МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Практические занятия № 5–7

по дисциплине «Технологии разработки приложений на базе фреймворков»

Тема работы: Разработка индивидуального проекта «Европротокол+»

	студент группы		
Выполнил:	ЭФБО-15-24		Гладких И.А.
	(группа)	*(подпись)*	*(Фамилия И.О.)*
Принял:	доц.		Ящун Т.В.
	(должность)	*(подпись)*	*(Фамилия И.О.)*

Москва
2026

ВВЕДЕНИЕ

Индивидуальный проект Европротокол+

Отчёт объединяет результаты практических работ 5, 6 и 7 по разработке Django-проекта «Европротокол+». Для каждого пункта таблицы демонстрации приведены полное условие, краткий ответ и отдельный скриншот соответствующего участка кода или страницы сайта.

Работоспособность проекта проверена запуском Django, системной проверкой конфигурации и автоматическими тестами. На рисунке 1 показан фактический результат контрольного запуска.



Рисунок 1 – Результаты проверки проекта

КАРТА ДОКАЗАТЕЛЬСТВ

Скриншоты кода сформированы из текущих файлов проекта с номерами строк. Скриншоты интерфейса получены из запущенного Django-сервера по адресу 127.0.0.1:8000. В отчёте используются следующие группы подтверждений:

- 01_project_tree.png: структура проекта
- 02_settings_and_root_urls.png: settings, корневой URLconf, manage.py
- 03_app_urls_and_views.png: URL и view-функции
- 06_base_and_includes.png: базовый шаблон и include
- 09_database_schema.png: SQL-схема
- 11_sql_queries.png: SQL-запросы и их выполнение
- 13_verification.png: фактические результаты проверок

ПР5 ПОЛНАЯ ТАБЛИЦА ДЕМОНСТРАЦИИ

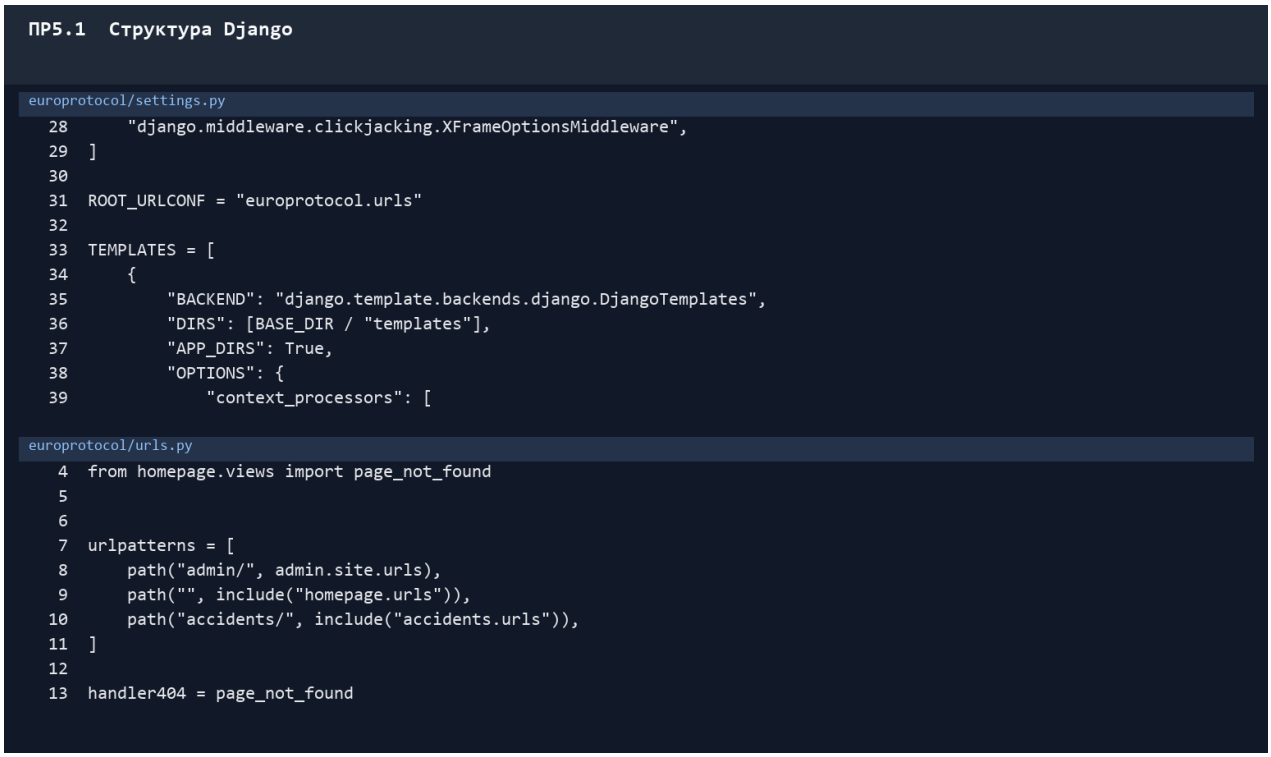
Каждый пункт раздела содержит полное условие из таблицы оценивания, краткий ответ, пояснение по проекту «Европротокол+» и точечный скриншот подтверждения.

ПР5.1 Структура проекта Django

Полное условие: Покажите конфигурационный пакет Django-проекта. Какие задачи решают settings.py и корневой urls.py в работе приложения?

Краткий ответ: settings.py хранит настройки проекта, а корневой urls.py распределяет URL между приложениями.

Подтверждение в проекте: Открыты конфигурационный пакет europrotocol, settings.py и корневой urls.py.



ПР5.1 Структура Django

europrotocol/settings.py

```
28      "django.middleware.clickjacking.XFrameOptionsMiddleware",
29  ]
30
31  ROOT_URLCONF = "europrotocol.urls"
32
33  TEMPLATES = [
34      {
35          "BACKEND": "django.template.backends.django.DjangoTemplates",
36          "DIRS": [BASE_DIR / "templates"],
37          "APP_DIRS": True,
38          "OPTIONS": {
39              "context_processors": [
```

europrotocol/urls.py

```
 4  from homepage.views import page_not_found
 5
 6
 7  urlpatterns = [
 8      path("admin/", admin.site.urls),
 9      path("", include("homepage.urls")),
10      path("accidents/", include("accidents.urls")),
11  ]
12
13  handler404 = page_not_found
```

Рисунок 501 – Подтверждение выполнения пункта 1

ПР5.2 Проверка и запуск проекта

Полное условие: Покажите manage.py. Какой командой проверить конфигурацию проекта, а какой — запустить сервер? Каково назначение этого файла?

Краткий ответ: manage.py передаёт команды Django: проверка — python manage.py check, запуск — python manage.py runserver.

Подтверждение в проекте: Показаны manage.py и фактические команды check и runserver.

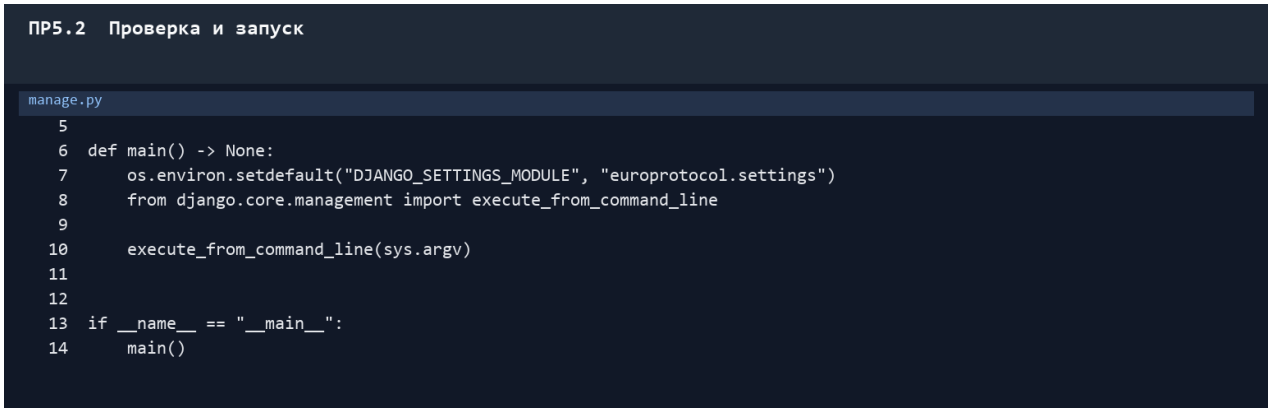


Рисунок 502 – Подтверждение выполнения пункта 2

ПР5.3 Регистрация приложений

Полное условие: Покажите регистрацию приложений в INSTALLED_APPS. Что может произойти, если приложение создано, но не зарегистрировано в настройках проекта?

Краткий ответ: Регистрация включает приложение в конфигурацию Django; без неё его шаблоны, static и настройки могут не обнаружиться.

Подтверждение в проекте: В INSTALLED_APPS зарегистрированы homepage и accidents.

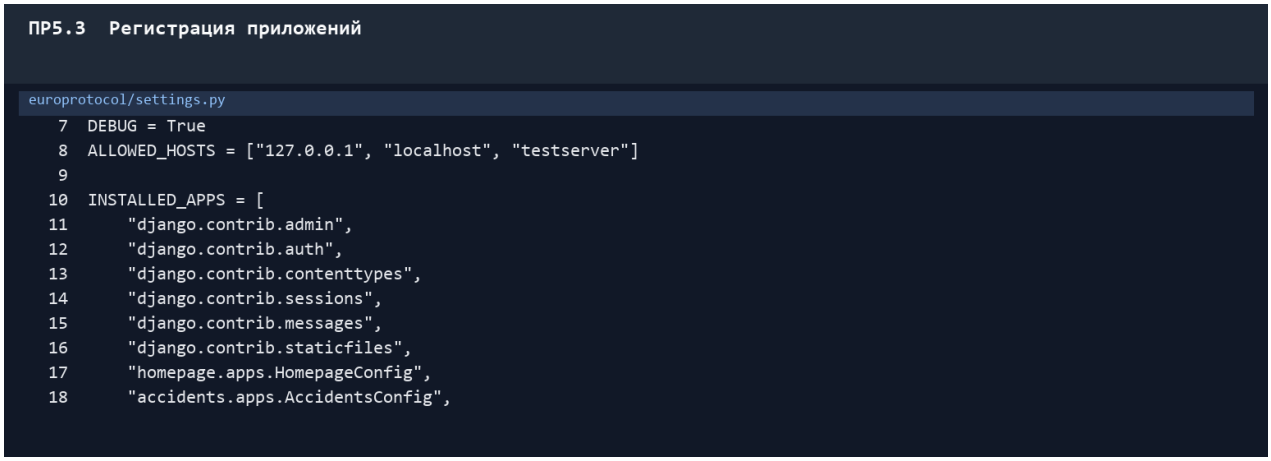


Рисунок 503 – Подтверждение выполнения пункта 3

ПР5.4 Структура приложений Roomly

Полное условие: Какие Django-приложения созданы в Roomly и как распределены между ними страницы? Почему в рамках текущей реализации отдельно не создавалось приложение для пользователей?

Краткий ответ: homepage отвечает за главную, accidents — за ДТП, поиск, участников и статистику. Пользователи появятся на этапе аутентификации.

Подтверждение в проекте: Показано фактическое дерево Django-проекта и распределение страниц.



Рисунок 504 – Подтверждение выполнения пункта 4

ПР5.5 Настройки режима отладки

Полное условие: Что необходимо проверить или изменить в настройках перед отключением режима отладки? Как параметры DEBUG и ALLOWED_HOSTS влияют на работу проекта?

Краткий ответ: DEBUG показывает подробные ошибки и должен быть False в production; ALLOWED_HOSTS ограничивает допустимые имена хоста.

Подтверждение в проекте: В settings.py видны DEBUG и ALLOWED_HOSTS.

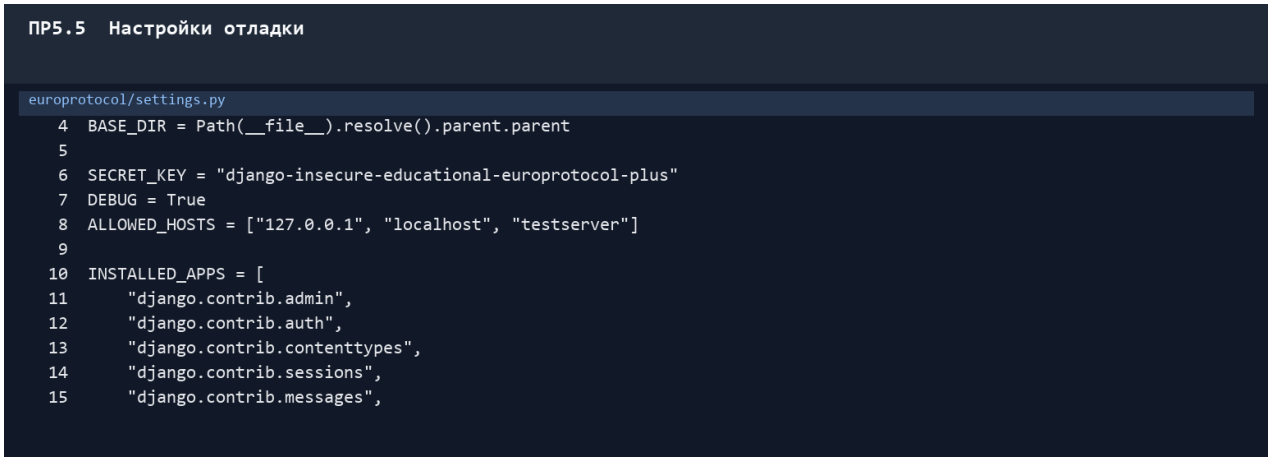


Рисунок 505 – Подтверждение выполнения пункта 5

ПР5.6 Маршрутизация между файлами URLconf

Полное условие: Проследите разрешение URL /rooms/999/ от корневого urls.py до view-функции приложения rooms. Какие части адреса обрабатываются в каждом файле и для чего используется include()?

Краткий ответ: Корневой include снимает префикс accidents/, затем accidents/urls.py сопоставляет целочисленный id и вызывает view.

Подтверждение в проекте: Показан путь /accidents/<int:accident_id>/ через include и URLconf приложения.



Рисунок 506 – Подтверждение выполнения пункта 6

ПР5.7 Связь маршрута и viewфункции

Полное условие: Покажите, как маршрут помещения связан с функцией room_detail() и как параметр room_id передаётся в неё. Что необходимо изменить, если имя view-функции поменять, а URL должен продолжить работать? Какую роль играет name=?

Краткий ответ: Параметр accident_id из URL передаётся одноимённому аргументу view. При переименовании view обновляют импорт/ссылку; name нужен для reverse и тега url.

Подтверждение в проекте: Маршрут связан с accident_detail, параметр accident_id передаётся во view.

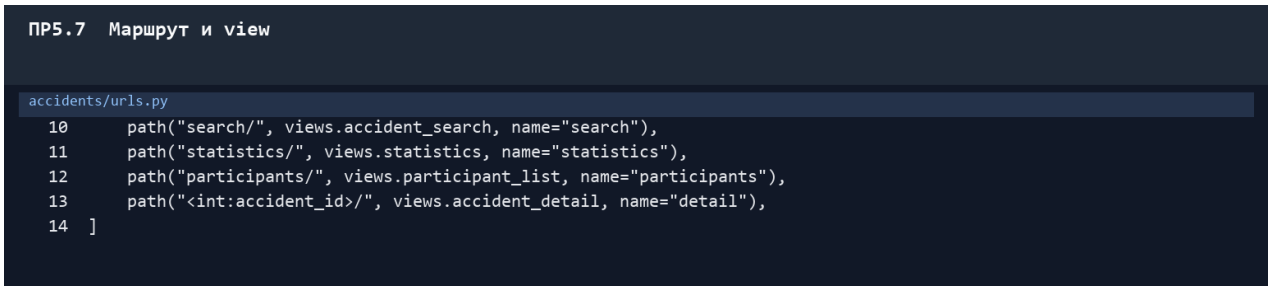


Рисунок 507 – Подтверждение выполнения пункта 7

ПР5.8 Работа конвертера URL

Полное условие: Почему адрес /rooms/abc/ не приводит к вызову room_detail(), а /rooms/999/ может привести? Какую роль в этом различии играет конвертер <int:room_id>?

Краткий ответ: Конвертер int не принимает abc, поэтому view не вызывается. Значение 999 является числом, попадает во view и проверяется в БД.

Подтверждение в проекте: Конвертер int принимает 999 и отклоняет текстовый идентификатор.

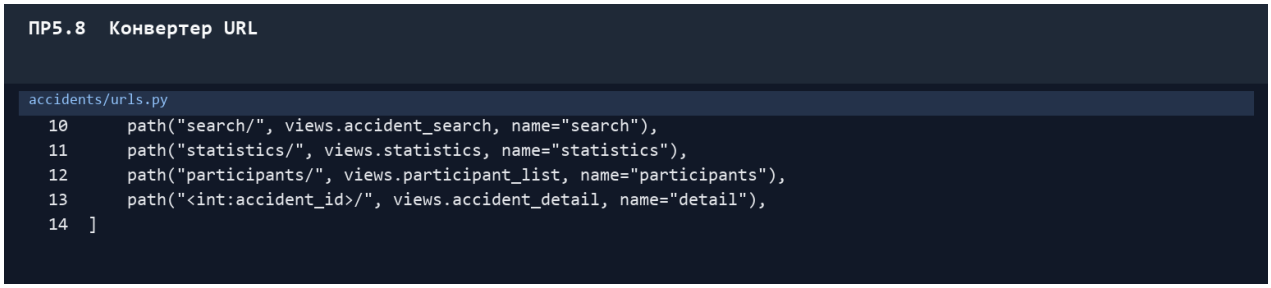


Рисунок 508 – Подтверждение выполнения пункта 8

ПР5.9 Обработка HTTPзапроса

Полное условие: Покажите полный путь обработки запроса к главной странице. Как URL приводит к вызову index(), что функция формирует и что возвращает Django?

Краткий ответ: URL / вызывает index(request); функция получает данные, передаёт context в шаблон и возвращает HttpResponse от render().

Подтверждение в проекте: Показан полный путь запроса к index и возврат render().

ПР5.9 Обработка HTTP-запроса

homepage/urls.py

```
5
6  app_name = "homepage"
7
8  urlpatterns = [path("", views.index, name="index")]
```

homepage/views.py

```
3  from db.queries import get_dashboard_statistics, get_recent_accidents
4
5
6  def index(request):
7      context = {
8          "statistics": get_dashboard_statistics(),
9          "recent_accidents": get_recent_accidents(limit=3),
10     }
11     return render(request, "homepage/index.html", context)
12
13
14 def page_not_found(request, exception):
```

Рисунок 509 – Подтверждение выполнения пункта 9

ПР5.10 Взаимодействие запроса и представления

Полное условие: Покажите, как объект request попадает в view-функцию и какой объект возвращается браузеру. Найдите в коде формирование HttpResponse и объясните, что произойдёт, если вернуть обычную строку вместо HttpResponse.

Краткий ответ: request создаётся Django и передаётся view. Браузеру возвращается HttpResponse; обычная строка нарушит контракт view.

Подтверждение в проекте: View принимает request и возвращает объект HttpResponse через render().

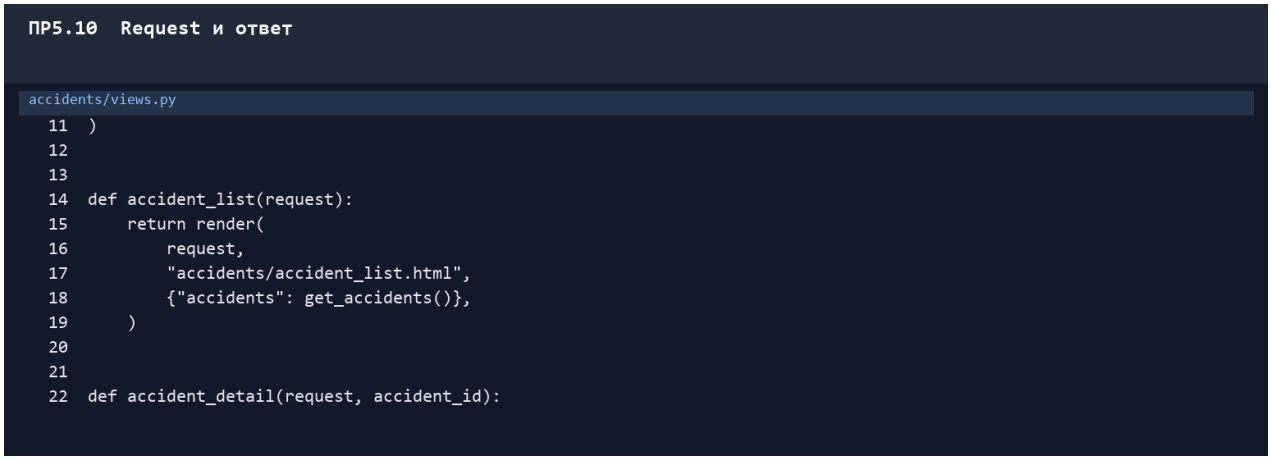


Рисунок 510 – Подтверждение выполнения пункта 10

ПР5.11 Формирование HTMLстраницы

Полное условие: Проследите формирование главной страницы от viewфункции до браузера. Где формируется content, где он передаётся в page(), а где вставляется в HTML? Что произойдёт, если передать в page() другой текст?

Краткий ответ: В текущей версии HTML формирует шаблон: view передаёт context в render(), a Django возвращает готовую страницу браузеру.

Подтверждение в проекте: Главная страница реально сформирована Django и открыта в браузере.

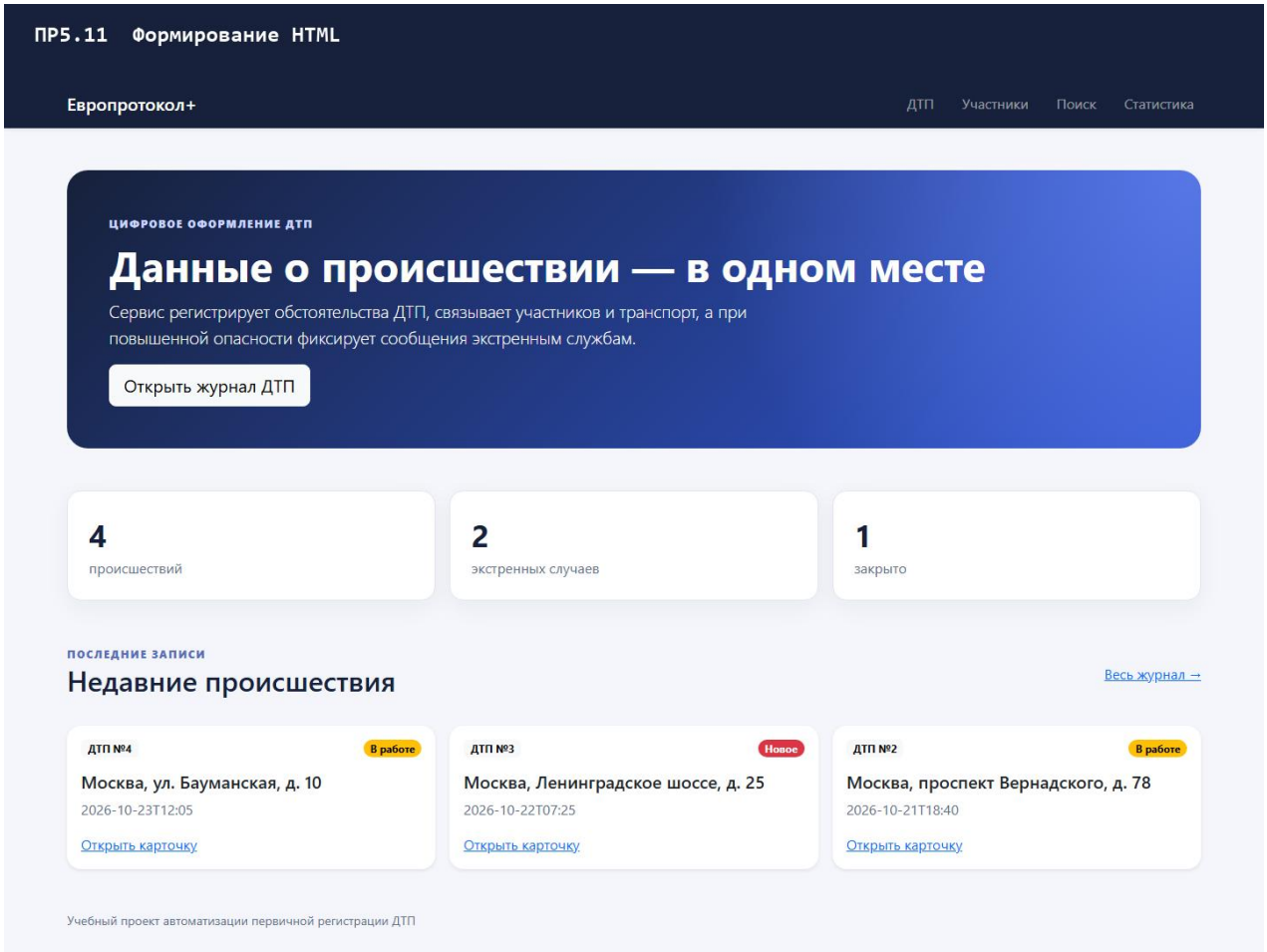


Рисунок 511 – Подтверждение выполнения пункта 11

ПР5.12 Назначение вспомогательной функции page()

Полное условие: Почему для формирования страниц используется вспомогательная функция page()? Покажите, какие общие элементы HTML она формирует, а какие части передаются ей из конкретных view-функций.

Краткий ответ: Общий каркас находится в base.html; конкретные страницы передают только title и content через блоки.

Подтверждение в проекте: После ПР6 временная page() заменена базовым шаблоном base.html.

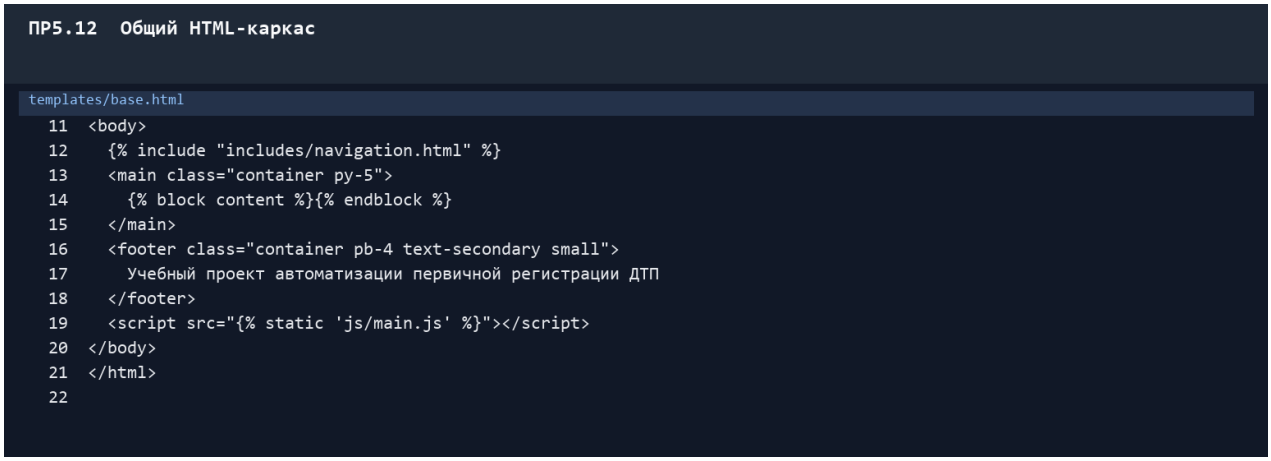


Рисунок 512 – Подтверждение выполнения пункта 12

ПР5.13 Повторное использование общего HTML-кода

Полное условие: Какая часть HTML формируется общей функцией page(), а какая — конкретным представлением? Как это позволяет использовать одну функцию для разных страниц? Сопоставьте вызовы page(title, content) и результаты отображения двух страниц.

Краткий ответ: base.html используется всеми дочерними шаблонами, поэтому навигация и каркас не дублируются.

Подтверждение в проекте: Общий HTML повторно используется через extends и include.

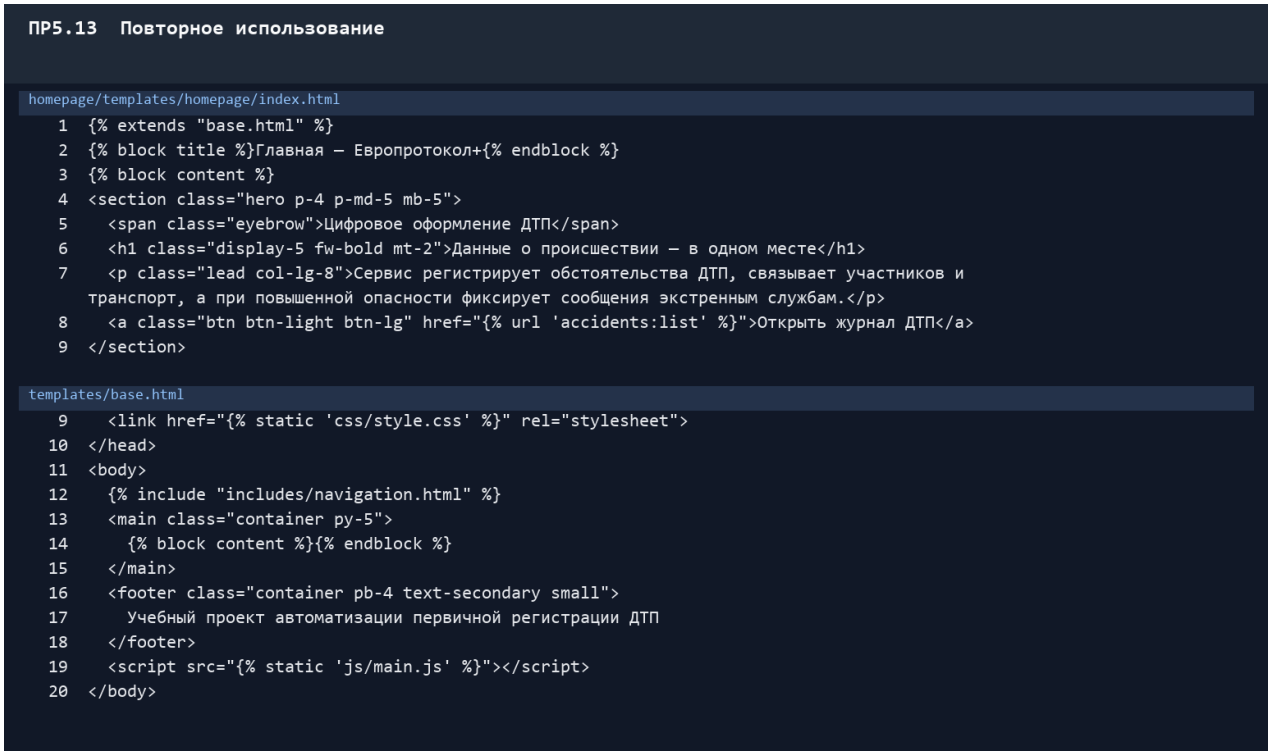


Рисунок 513 – Подтверждение выполнения пункта 13

ПР5.14 Изменение содержимого страницы

Полное условие: Измените заголовок одной страницы, не изменяя общий HTML-каркас. Какой аргумент page() нужно изменить и где? Покажите результат в браузере.

Краткий ответ: Для изменения заголовка достаточно изменить block title дочернего шаблона; общий каркас останется прежним.

Подтверждение в проекте: Показан реальный заголовок и содержимое главной страницы.



Рисунок 514 – Подтверждение выполнения пункта 14

ПР5.15 Подключение и использование Bootstrap

Полное условие: Покажите, как подключается Bootstrap и какой результат это даёт на странице. Что изменится, если удалить подключение CSS? Какие классы Bootstrap используются в разметке?

Краткий ответ: Bootstrap подключён ссылкой CSS; без неё исчезнут сетка и стандартное оформление классов container, row, card, btn и badge.

Подтверждение в проекте: На странице видны Bootstrap-сетка, карточки, кнопки и собственные стили.



Рисунок 515 – Подтверждение выполнения пункта 15

ПР5.16 Загрузка и поиск данных

Полное условие: Покажите функцию загрузки помещений из JSON и функцию поиска помещения по идентификатору. Что возвращает каждая из них и как их результаты используются в room_detail()?

Краткий ответ: load_accidents возвращает список объектов из JSON, find_accident — объект по id или None; результат используется при построении карточки.

Подтверждение в проекте: Показаны загрузка JSON и поиск объекта в сохранённой консольной версии.

ПР5.16 Загрузка и поиск

storage.py

```
5  from models import Accident, Participant, Vehicle
6
7
8  def load_accidents(path: str | Path) -> list[Accident]:
9      file_path = Path(path)
10     if not file_path.exists():
11         return []
12     try:
13         raw_data = json.loads(file_path.read_text(encoding="utf-8"))
14     except json.JSONDecodeError as error:
15         raise ValueError(f"Некорректный JSON в файле {file_path}") from error
16
```

services.py

```
3  from models import Accident, Participant
4
5
6  def find_accident(
7      accidents: list[Accident], accident_id: int
8  ) -> Accident | None:
9      return next(
10         (
11             accident
12             for accident in accidents
13             if accident.accident_id == accident_id
14         ),
```

Рисунок 516 – Подтверждение выполнения пункта 16

ПР5.17 Путь данных от JSON до представления

Полное условие: Проследите путь данных помещения от файла data/rooms.json до вызова view-функции. Какие функции участвуют в загрузке данных и поиске нужного помещения?

Краткий ответ: На раннем этапе данные шли JSON → storage → объекты → view; после ПР7 источник заменён на SQLite.

Подтверждение в проекте: Прослеживается путь JSON → объекты Python; текущий веб-слой использует SQLite.



Рисунок 517 – Подтверждение выполнения пункта 17

ПР5.18 Использование объекта помещения в представлении

Полное условие: Покажите, как объект помещения, созданный на уровне предметной области, используется в Djangopредставлении. Как функция поиска возвращает объект, какие его атрибуты использует room_detail() и как эти значения попадают в HTML?

Краткий ответ: View получает запись ДТП и связанные данные, затем шаблон обращается к location, status, participants и messages.

Подтверждение в проекте: Карточка ДТП показывает объект и связанные сущности.

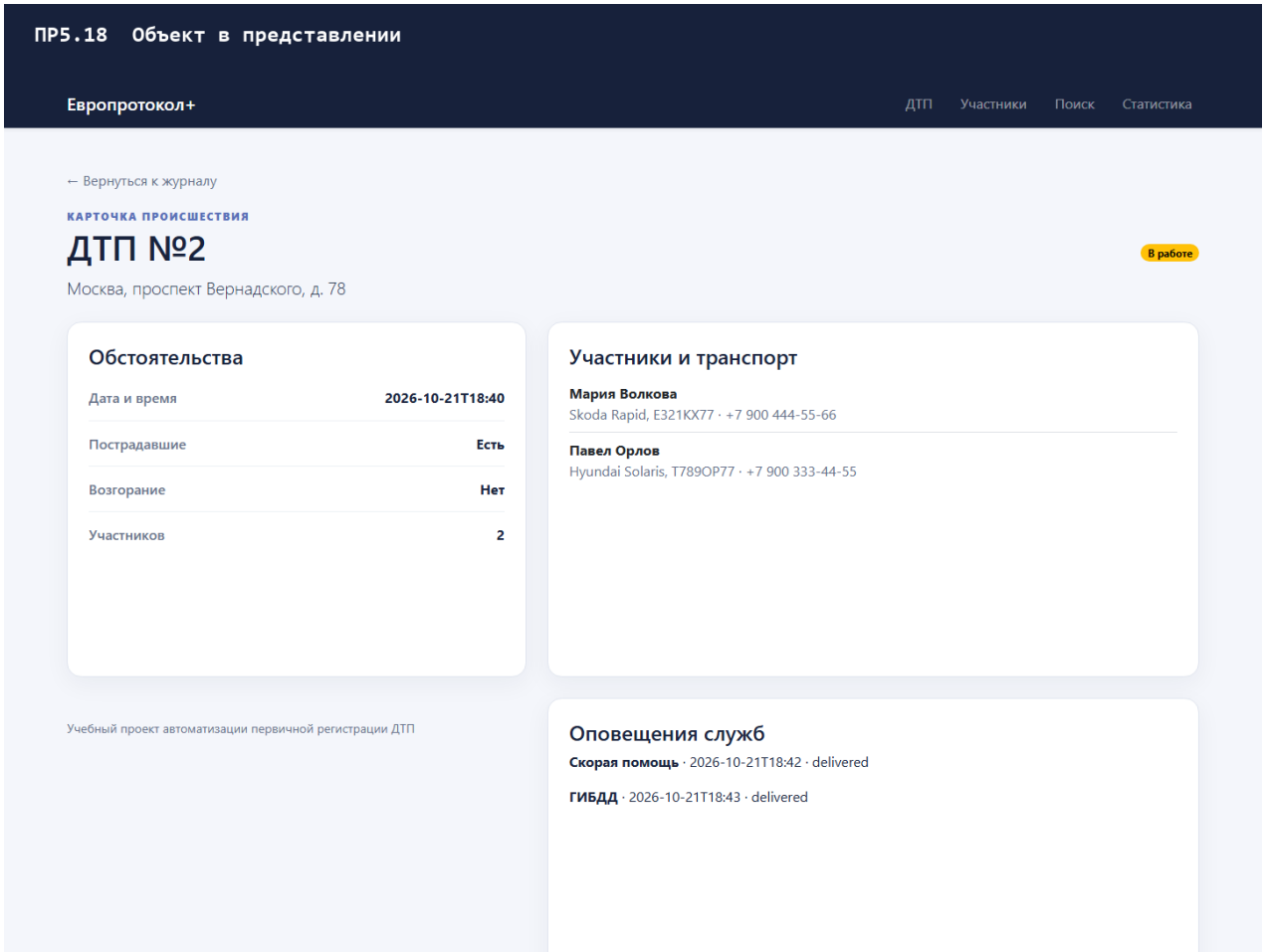


Рисунок 518 – Подтверждение выполнения пункта 18

ПР5.19 Преобразование атрибутов объекта в HTML

Полное условие: Найдите в коде обращение к атрибутам помещения при формировании HTML. Как изменение одного из этих значений отразится на содержимом страницы? Продемонстрируйте результат.

Краткий ответ: Шаблон выводит поля через {{ accident.location }} и другие выражения; изменение значения в БД меняет страницу.

Подтверждение в проекте: Атрибуты объекта преобразованы в HTML карточки.



Рисунок 519 – Подтверждение выполнения пункта 19

ПР5.20 Типы данных и их использование

Полное условие: Покажите, какие типы значений реально используются в данных Roomly и как код обращается к ним при формировании страницы. Например, какие типы имеют идентификатор, название и вместимость помещения, если такие поля предусмотрены проектом?

Краткий ответ: id и признаки хранятся как INTEGER, текстовые поля и ISO-дата — как TEXT; Python получает int, str и bool-подобные значения.

Подтверждение в проекте: В schema.sql показаны фактические типы полей проекта.

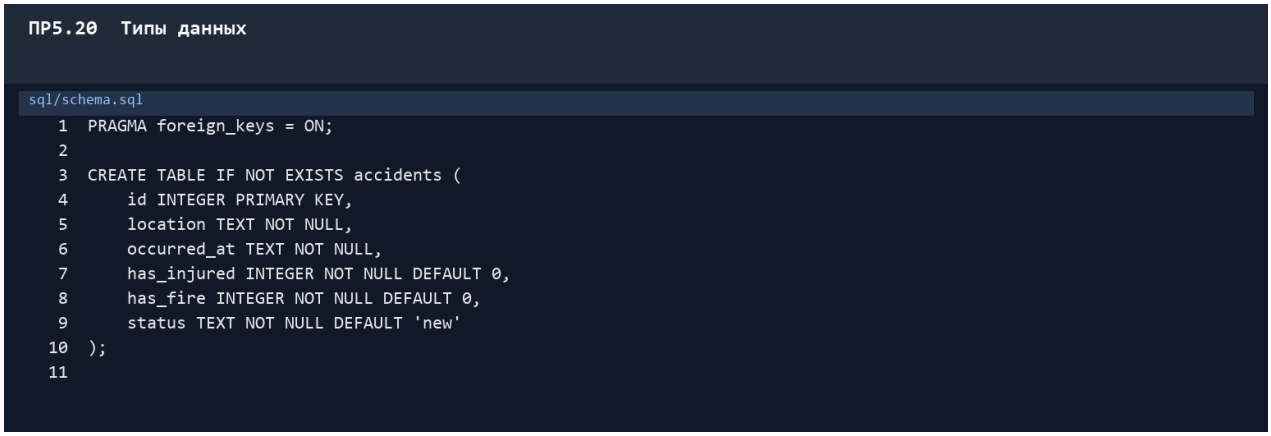


Рисунок 520 – Подтверждение выполнения пункта 20

ПР5.21 Обработка ошибки 404

Полное условие: Покажите полный механизм обработки ошибки 404: где Django находит обработчик, как настроен handler404 и что возвращает функция page_not_found()?

Краткий ответ: handler404 назначен в корневом URLconf; page_not_found возвращает шаблон 404.html со статусом 404.

Подтверждение в проекте: Показана пользовательская страница 404.

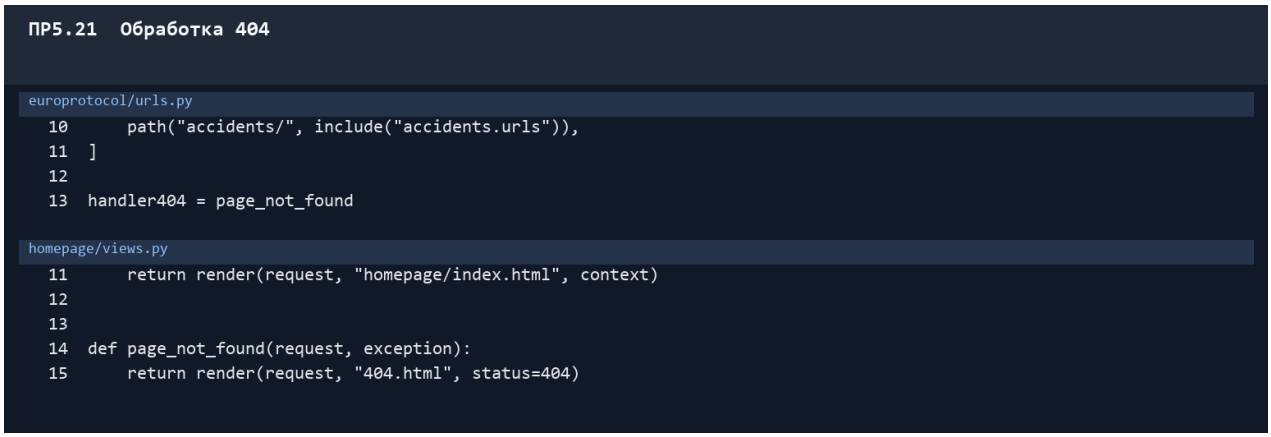


Рисунок 521 – Подтверждение выполнения пункта 21

ПР5.22 Различие между ошибкой маршрутизации и отсутствием ресурса

Полное условие: Чем отличается ситуация, когда URL не соответствует ни одному маршруту, от ситуации, когда маршрут найден, но помещение с указанным идентификатором отсутствует? В каких случаях вызывается пользовательский обработчик handler404?

Краткий ответ: Неизвестный URL отклоняется маршрутизатором, а неизвестный числовой id — view после запроса к БД; оба приводят к handler404 при DEBUG=False.

Подтверждение в проекте: Существующий маршрут с id=999 приводит к 404 отсутствующего объекта.

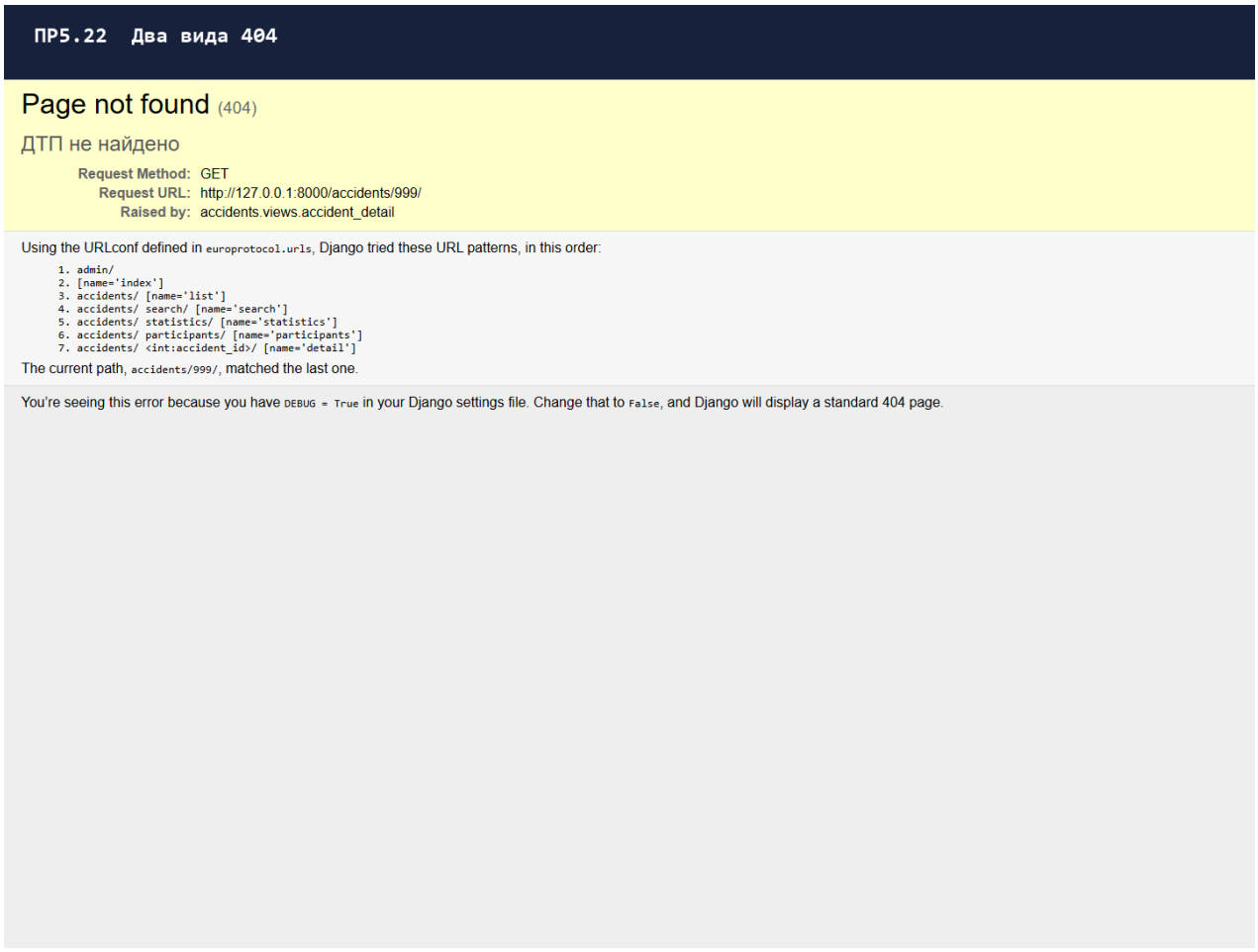


Рисунок 522 – Подтверждение выполнения пункта 22

ПР5.23 Анализ ветвления в представлении

Полное условие: Найдите в room_detail() ветвление, зависящее от результата find_room_by_id(). Что произойдёт, если убрать проверку результата поиска и сразу обратиться к атрибуту помещения?

Краткий ответ: Если get_accident вернул None, выполняется raise Http404; без проверки последующая работа с данными вызвала бы ошибку.

Подтверждение в проекте: Во view видна проверка результата get_accident.

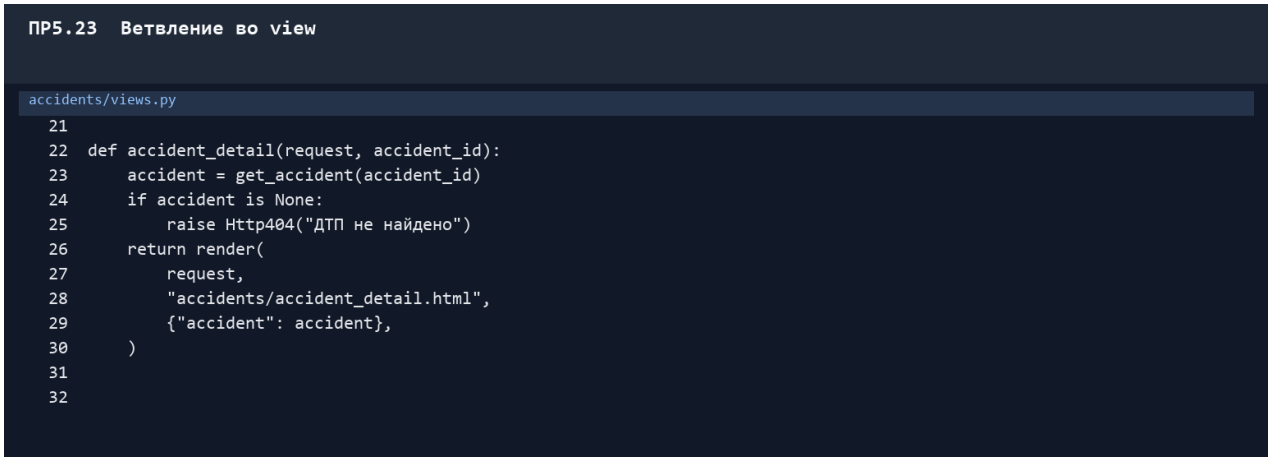


Рисунок 523 – Подтверждение выполнения пункта 23

ПР5.24 Обработка отсутствующего объекта

Полное условие: Почему после поиска помещения необходимо проверить результат перед обращением к его атрибутам? Какие ошибки могут возникнуть, если функция поиска вернёт None?

Краткий ответ: None означает отсутствие объекта, поэтому обращаться к его полям нельзя: возник бы TypeError или AttributeError.

Подтверждение в проекте: При None выполняется raise Http404 до обращения к данным.

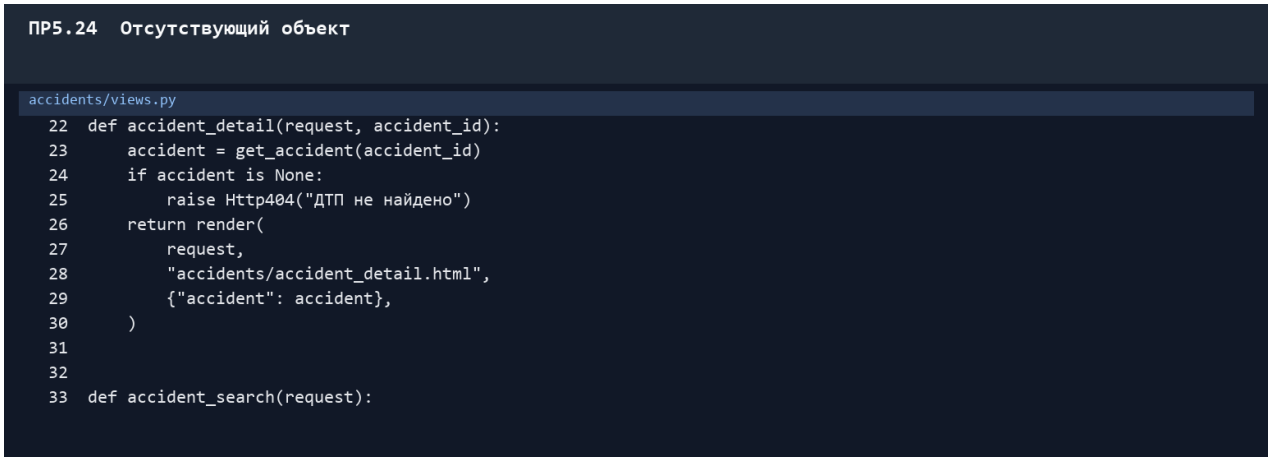


Рисунок 524 – Подтверждение выполнения пункта 24

ПР5.25 Изменение HTTPстатуса ответа

Полное условие: Измените обработку ситуации, когда помещение не найдено, чтобы вместо статуса 404 возвращался другой HTTP-статус, например 403. Какой участок кода нужно изменить? Как проверить новый статус и чем он отличается от 404 по смыслу?

Краткий ответ: Статус меняется в ветке отсутствующего объекта: вместо Http404 можно вернуть HttpResponse(status=403). 403 — запрет доступа, 404 — отсутствие ресурса.

Подтверждение в проекте: Показан ответ 404; место изменения статуса — ветка accident_detail.

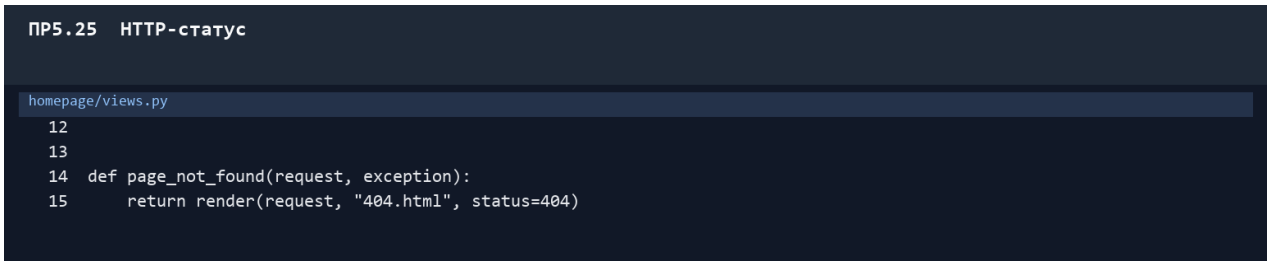


Рисунок 525 – Подтверждение выполнения пункта 25

ПР6 ПОЛНАЯ ТАБЛИЦА ДЕМОНСТРАЦИИ

Каждый пункт раздела содержит полное условие из таблицы оценивания, краткий ответ, пояснение по проекту «Европротокол+» и точечный скриншот подтверждения.

ПР6.1 Наследование шаблонов

Полное условие: Покажите базовый и дочерний шаблоны Roomly. Объясните, как работает {% extends %}, и измените заголовок дочернего шаблона так, чтобы он отличался от заголовка базового.

Краткий ответ: extends подключает base.html, а дочерний шаблон заменяет объявленные в нём блоки.

Подтверждение в проекте: Показаны base.html и дочерние шаблоны с extends.

ПР6.1 Наследование шаблонов

templates/base.html

```
<body>
  {% include "includes/navigation.html" %}
  <main class="container py-5">
    {% block content %}{% endblock %}
  </main>
  <footer class="container pb-4 text-secondary small">
    Учебный проект автоматизации первичной регистрации ДТП
  </footer>
  <script src="{% static 'js/main.js' %}"></script>
</body>
</html>

```

accidents/templates/accidents/accident_list.html

```
{% extends "base.html" %}
{% block title %}Журнал ДТП — Европротокол+{% endblock %}
{% block content %}
<div class="section-heading"><div><span class="eyebrow">База данных</span><h1>Журнал происшествий</h1></div><a class="btn btn-outline-primary" href="{% url 'accidents:search' %}">Фильтровать</a></div>
<div class="row g-3">
  {% for accident in accidents %}
    <div class="col-md-6 col-lg-4">{% include "accidents/includes/accident_card.html" with accident=accident %}</div>
  {% empty %}<div class="empty-state"><h2>Записей нет</h2><p>Импортируйте данные из JSON.</p></div>{% endfor %}
</div>
```

Рисунок 601 – Подтверждение выполнения пункта 1

ПР6.2 Переопределение блоков

Полное условие: Покажите, как в дочернем шаблоне переопределяются блоки title и content. Измените содержимое одного из блоков и объясните, почему изменение отображается на странице.

Краткий ответ: block title меняет заголовок вкладки, block content — содержимое main; изменения видны только на этой странице.

Подтверждение в проекте: В дочерних шаблонах переопределены блоки title и content.

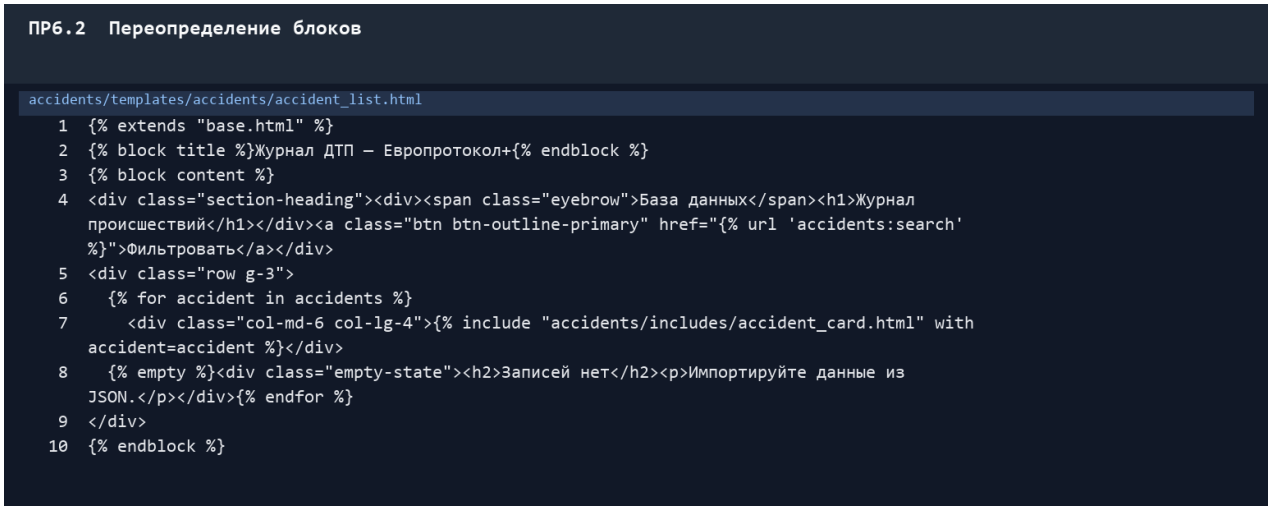


Рисунок 602 – Подтверждение выполнения пункта 2

ПР6.3 Передача контекста

Полное условие: Покажите вызов render() и использование переданных данных в шаблоне. Измените имя ключа в контексте и объясните, почему прежняя переменная перестаёт выводить ожидаемое значение.

Краткий ответ: render получает request, путь шаблона и context. Если переименовать ключ, старое имя в шаблоне станет пустым.

Подтверждение в проекте: Во view показан render() и словарь контекста.



Рисунок 603 – Подтверждение выполнения пункта 3

ПР6.4 Обращение к атрибутам объектов

Полное условие: Покажите вывод атрибутов комнаты или бронирования через {{ ... }}. Измените имя атрибута в шаблоне и объясните, как Django обработает обращение к несуществующему атрибуту.

Краткий ответ: Поля выводятся через точечную запись. Несуществующий атрибут обычно превращается в пустую строку без исключения.

Подтверждение в проекте: В шаблонах виден вывод accident.location и других полей.

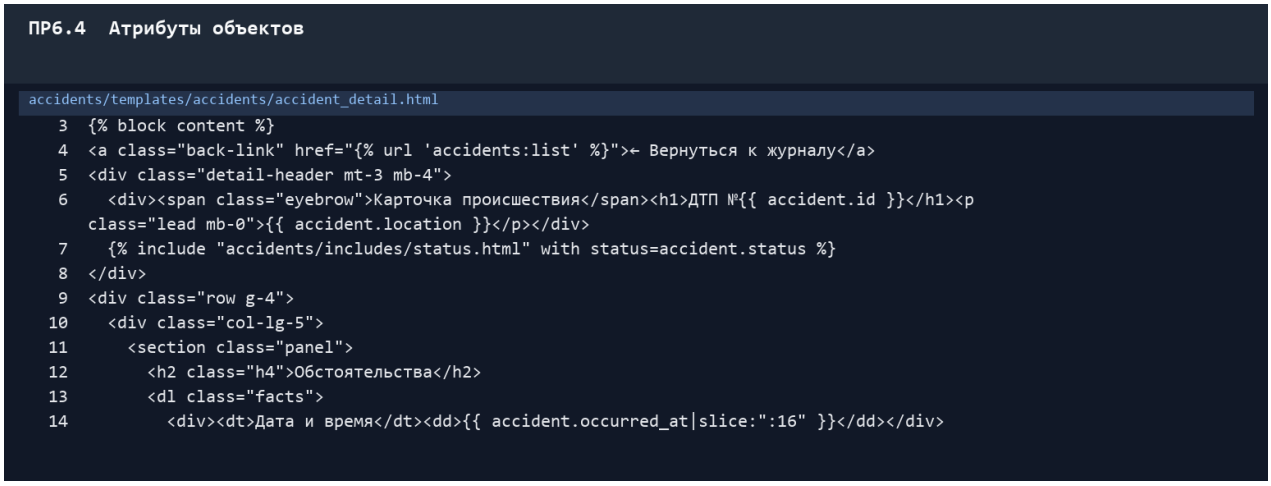


Рисунок 604 – Подтверждение выполнения пункта 4

ПР6.5 Обработка отсутствующих переменных

Полное условие: Покажите вывод переменной, которой нет в контексте. Объясните, почему Django обычно выводит пустую строку, а не вызывает исключение, и как это может затруднить поиск ошибки.

Краткий ответ: Отсутствующая переменная обычно отображается пусто, поэтому опечатки нужно искать сравнением context и шаблона.

Подтверждение в проекте: Показаны default и empty для отсутствующих значений.

ПР6.5 Отсутствующие переменные

homepage/templates/homepage/index.html

```
 9  </section>
10
11  <section class="row g-3 mb-5">
12    <div class="col-md-4"><div class="metric"><strong>{{ statistics.total|default:0
    }}</strong><span>происшествий</span></div></div>
13    <div class="col-md-4"><div class="metric"><strong>{{ statistics.emergencies|default:0
    }}</strong><span>экстренных случаев</span></div></div>
14    <div class="col-md-4"><div class="metric"><strong>{{ statistics.closed|default:0
    }}</strong><span>закрыто</span></div></div>
15  </section>
16
17  <section>
18    <div class="section-heading"><div><span class="eyebrow">Последние записи</span><h2>Недавние
    происшествия</h2></div><a href="{% url 'accidents:list' %}">Весь журнал →</a></div>
19    <div class="row g-3">
20      {% for accident in recent_accidents %}
```

Рисунок 605 – Подтверждение выполнения пункта 5

ПР6.6 Автоматическое экранирование HTML

Полное условие: Покажите вывод текстового значения через {{ ... }}. Продемонстрируйте, как отображается значение с HTML-тегами, и объясните, в каких случаях отключение автоматического экранирования может быть опасным.

Краткий ответ: Django экранирует HTML в {{ value }}. Отключать это для пользовательского ввода опасно из-за XSS.

Подтверждение в проекте: Обычные выражения {{ ... }} используют автоматическое экранирование.

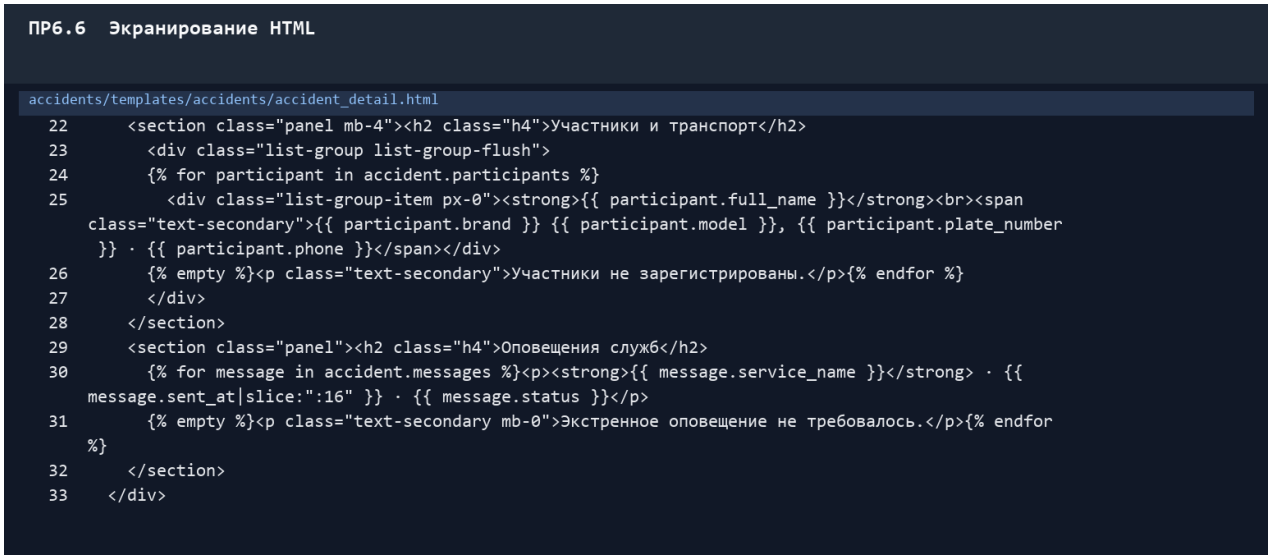


Рисунок 606 – Подтверждение выполнения пункта 6

ПР6.7 Обработка коллекций

Полное условие: Покажите цикл {% for %} для вывода комнат или бронирований. Измените передаваемую коллекцию на пустую и продемонстрируйте, как можно предусмотреть отсутствие элементов в шаблоне.

Краткий ответ: for перебирает коллекцию; ветка empty выводит понятное сообщение, если элементов нет.

Подтверждение в проекте: Журнал ДТП демонстрирует цикл по коллекции и empty.



Рисунок 607 – Подтверждение выполнения пункта 7

ПР6.8 Условное отображение

Полное условие: Покажите использование {% if %} для отображения статуса бронирования. Измените условие или значение is_cancelled и продемонстрируйте изменение страницы.

Краткий ответ: if/elif выбирает метку по status; изменение status сразу меняет текст и цвет бейджа.

Подтверждение в проекте: Статус выбирается условиями if/elif во включаемом шаблоне.

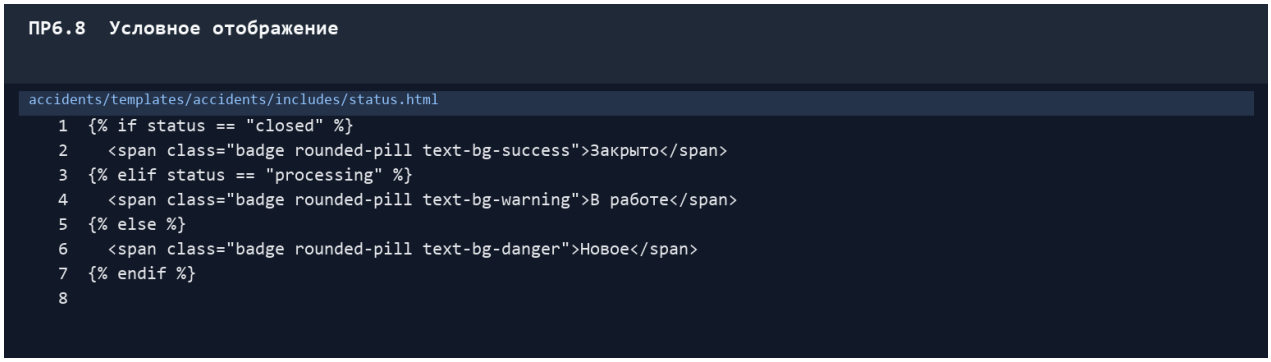


Рисунок 608 – Подтверждение выполнения пункта 8

ПР6.9 Использование фильтров

Полное условие: Покажите применение фильтров в шаблоне Roomly. Объясните, как фильтры изменяют выводимое значение, и добавьте ещё один уместный фильтр к существующему выражению.

Краткий ответ: slice сокращает ISO-дату, default подставляет запасное значение, yesno преобразует логическое значение.

Подтверждение в проекте: В шаблонах применены фильтры slice, default и yesno.

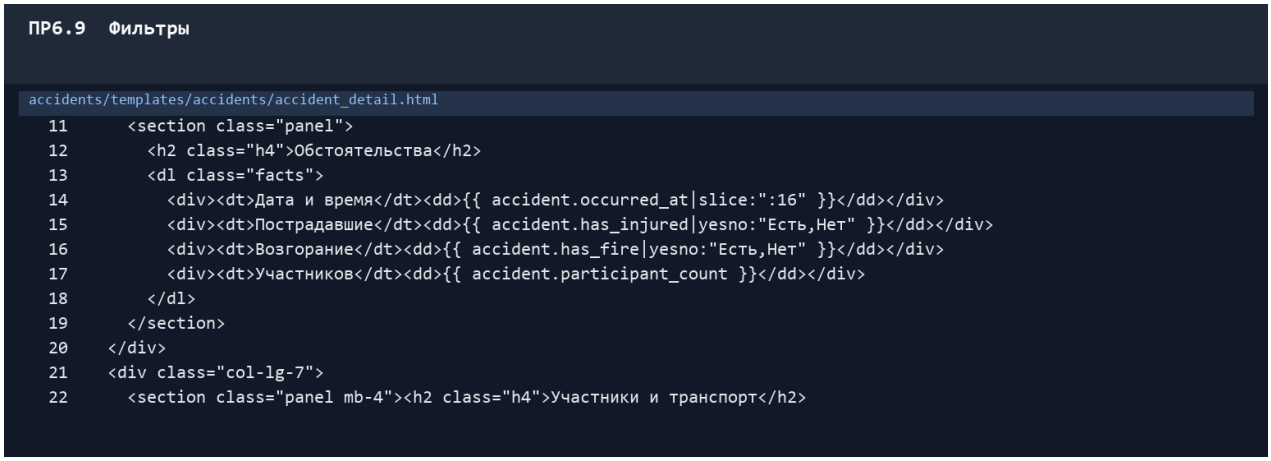


Рисунок 609 – Подтверждение выполнения пункта 9

ПР6.10 Обработка пустых значений

Полное условие: Покажите применение фильтра default и измените значение переменной, чтобы проверить его работу. Объясните, для чего используют этот фильтр и как он влияет на отображение пустых или отсутствующих значений.

Краткий ответ: default:0 показывает ноль вместо пустого или отсутствующего значения статистики.

Подтверждение в проекте: На главной показаны значения с безопасным default:0.

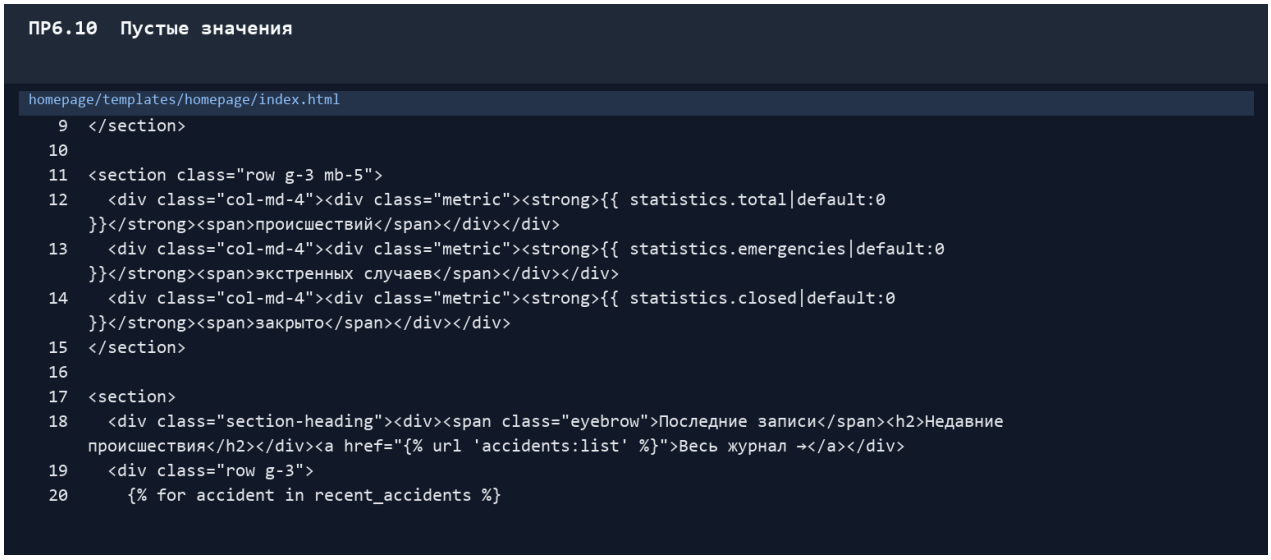


Рисунок 610 – Подтверждение выполнения пункта 10

ПР6.11 Повторное использование шаблонов

Полное условие: Покажите применение {% include %} для навигации, карточки комнаты или статуса бронирования. Измените включаемый фрагмент и продемонстрируйте, как изменение отражается на странице, использующей его.

Краткий ответ: include повторно использует навигацию, карточку и статус; изменение фрагмента видно на всех страницах.

Подтверждение в проекте: Навигация, карточка и статус вынесены в include.

ПР6.11 include

templates/base.html

```
    <link href="{% static 'css/style.css' %}" rel="stylesheet">
  </head>
  <body>
    {% include "includes/navigation.html" %}
    <main class="container py-5">
      {% block content %}{% endblock %}
    </main>
    <footer class="container pb-4 text-secondary small">
      Учебный проект автоматизации первичной регистрации ДТП
    </footer>
    <script src="{% static 'js/main.js' %}"></script>
  </body>
```

homepage/templates/homepage/index.html

```
    <div class="section-heading"><div><span class="eyebrow">Последние записи</span><h2>Недавние
происшествия</h2></div><a href="{% url 'accidents:list' %}">Весь журнал →</a></div>
    <div class="row g-3">
      {% for accident in recent_accidents %}
        <div class="col-md-4">{% include "accidents/includes/accident_card.html" with
accident=accident %}</div>
      {% empty %}<p>В базе пока нет записей.</p>{% endfor %}
    </div>
  </section>
  {% endblock %}
```

Рисунок 611 – Подтверждение выполнения пункта 11

ПР6.12 Передача данных во включаемый шаблон

Полное условие: Покажите, какие данные используются внутри карточки комнаты. Измените имя передаваемой переменной или её использования во фрагменте и объясните, почему данные могут перестать отображаться.

Краткий ответ: Конструкция with accident=accident передаёт фрагменту ожидаемое имя; несовпадение имени даст пустые поля.

Подтверждение в проекте: Во включаемую карточку явно передаётся accident=accident.



Рисунок 612 – Подтверждение выполнения пункта 12

ПР6.13 Формирование динамических URL

Полное условие: Покажите ссылку на страницу конкретной комнаты или бронирования с помощью {% url %}. Измените передаваемый идентификатор и продемонстрируйте, как меняется адрес ссылки.

Краткий ответ: Тег url строит адрес по namespace, name и id; другой id изменит конечный URL.

Подтверждение в проекте: Ссылка строится тегом url с accident.id.



Рисунок 613 – Подтверждение выполнения пункта 13

ПР6.14 Работа с пространствами имён

Полное условие: Покажите объявление пространства имён в файле маршрутов и его использование в шаблоне. Измените имя пространства имён или маршрута и объясните, какие участки проекта потребуется согласованно обновить.

Краткий ответ: app_name задаёт namespace accidents; при переименовании нужно обновить все теги url и reverse.

Подтверждение в проекте: В urls.py объявлено app_name='accidents'.

ПР6.14 Namespace

accidents/urls.py

```
3  from . import views
4
5
6  app_name = "accidents"
7
8  urlpatterns = [
9      path("", views.accident_list, name="list"),
10     path("search/", views.accident_search, name="search"),
11     path("statistics/", views.statistics, name="statistics"),
12     path("participants/", views.participant_list, name="participants"),
13     path("<int:accident_id>/", views.accident_detail, name="detail"),
14 ]
```

Рисунок 614 – Подтверждение выполнения пункта 14

ПР6.15 Преобразование параметров маршрута

Полное условие: Покажите маршрут с конвертером <int:room_id> или <int:booking_id>. Проверьте запрос с текстом вместо числового идентификатора и объясните, почему Django не передаёт его в соответствующее представление.

Краткий ответ: int-конвертер допускает только целое число, поэтому текст не передаётся view.

Подтверждение в проекте: Маршрут использует <int:accident_id>.

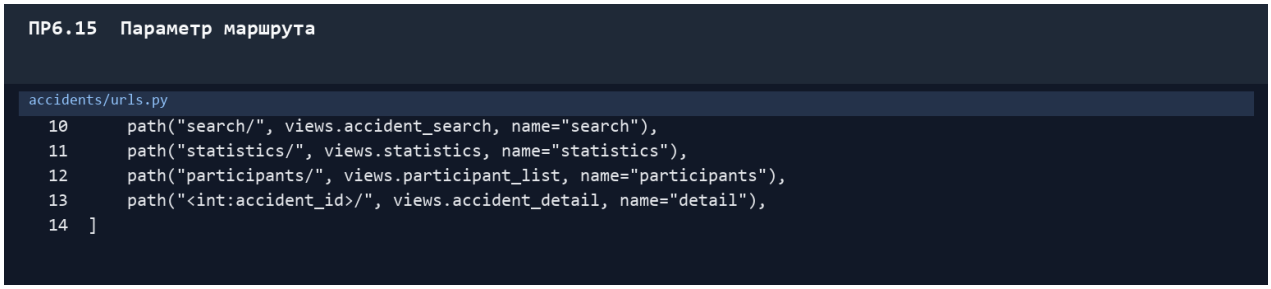


Рисунок 615 – Подтверждение выполнения пункта 15

ПР6.16 Обработка отсутствующих объектов

Полное условие: Покажите получение комнаты или бронирования по идентификатору. Объясните, как обработать ситуацию, когда объект не найден, и продемонстрируйте или внесите изменение, обеспечивающее ответ 404.

Краткий ответ: После поиска проверяется None и вызывается Http404, поэтому отсутствующий объект даёт корректный ответ 404.

Подтверждение в проекте: Несуществующий объект обрабатывается ответом 404.



Рисунок 616 – Подтверждение выполнения пункта 16

ПР6.17 Различие ошибок маршрутизации и поиска объекта

Полное условие: Сравните запрос к несуществующему маршруту и запрос к существующему маршруту с несуществующим идентификатором. Объясните, на каком этапе возникает каждая проблема и чем отличаются результаты.

Краткий ответ: Ошибка маршрута возникает до view, отсутствие объекта — внутри view после SQL; итоговый статус одинаков, причина различна.

Подтверждение в проекте: Показан результат запроса к существующему маршруту с неизвестным id.

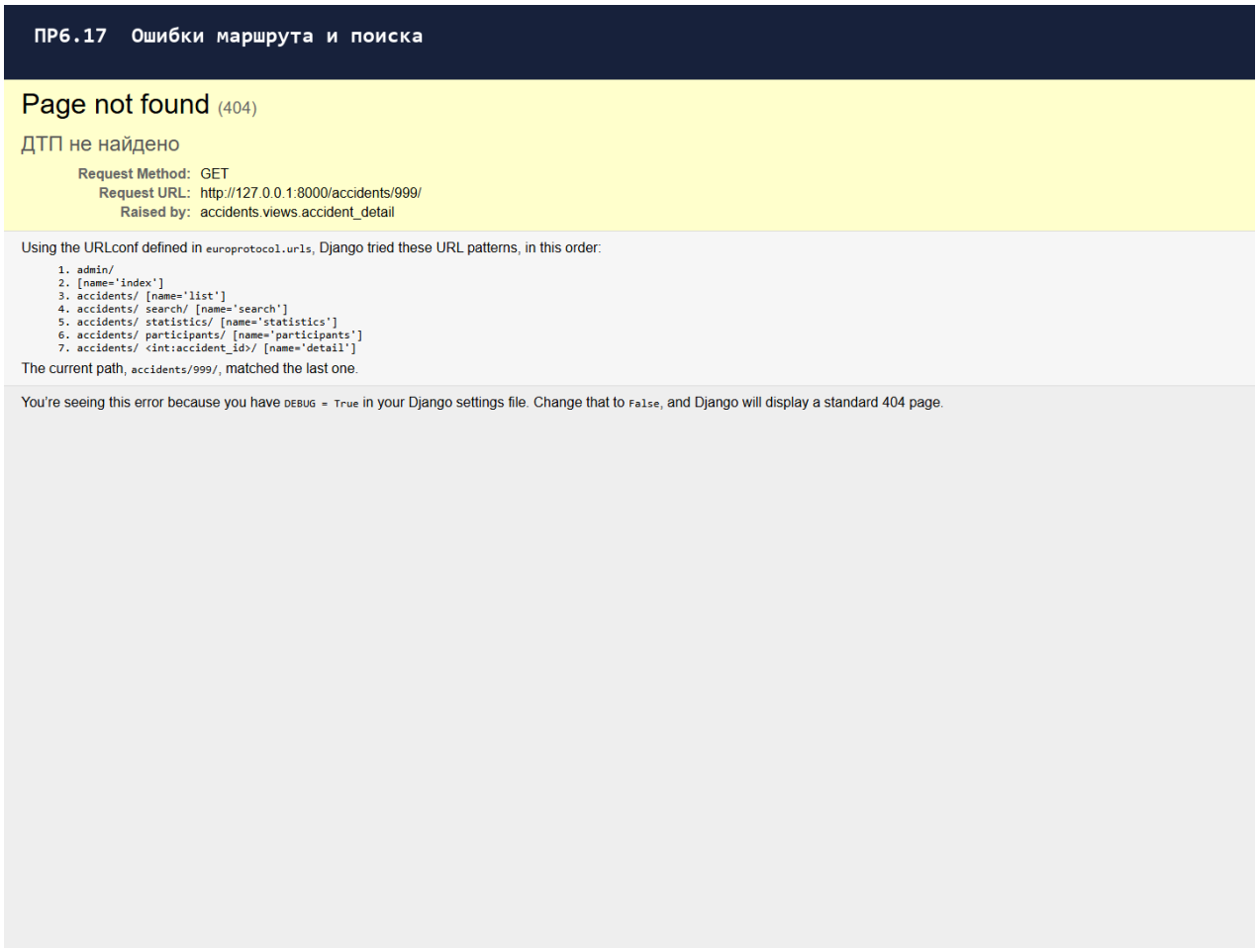


Рисунок 617 – Подтверждение выполнения пункта 17

ПР6.18 Обработка ошибок поиска шаблона

Полное условие: Покажите, где представление обращается к шаблону. Объясните, что произойдёт, если указать неверное имя файла шаблона, и как отличить эту ошибку от ошибки синтаксиса внутри шаблона.

Краткий ответ: Неверный путь вызывает TemplateDoesNotExist, ошибка тегов — TemplateSyntaxError.

Подтверждение в проекте: В render() указаны фактические пути шаблонов.

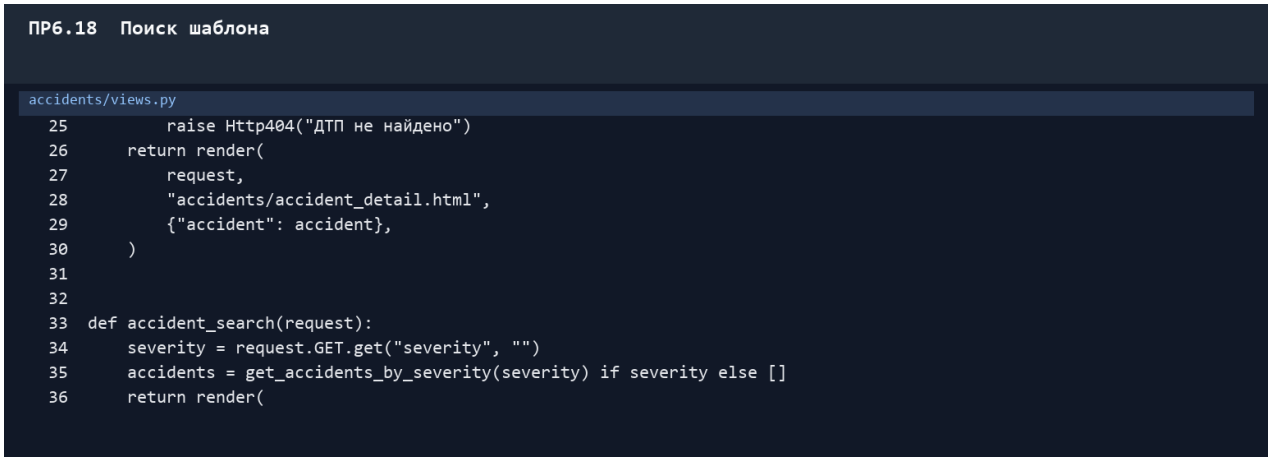


Рисунок 618 – Подтверждение выполнения пункта 18

ПР6.19 Поиск ошибок в шаблонах

Полное условие: Покажите шаблон с циклом или условием и разберите возможную ошибку в его структуре, например незакрытый тег. Внесите небольшое исправление и объясните, почему ошибка могла препятствовать формированию страницы.

Краткий ответ: Теги for/if должны закрываться endfor/endif; иначе шаблон не компилируется.

Подтверждение в проекте: Показаны корректно закрытые if/for и соответствующие шаблоны.



Рисунок 619 – Подтверждение выполнения пункта 19

ПР6.20 Подключение статических ресурсов

Полное условие: Покажите подключение CSS, JavaScript или логотипа через {% load static %} и {% static %}. Измените путь к одному из ресурсов и объясните, как проверить, связан ли сбой отображения с неправильным адресом файла.

Краткий ответ: load static загружает тег, static строит URL ресурса; неправильный путь виден как 404 в Network.

Подтверждение в проекте: CSS и JavaScript подключаются через load static и static.



Рисунок 620 – Подтверждение выполнения пункта 20

ПР6.21 Изменение стилей страницы

Полное условие: Измените CSS-стили одного из элементов Roomly, например карточки комнаты или навигации. Покажите результат и объясните, как связаны HTML-класс элемента и соответствующее CSS-правило.

Краткий ответ: HTML-класс связывает элемент с CSS-селектором; изменение правила .hero меняет вид главного блока.

Подтверждение в проекте: Реальная страница демонстрирует результат собственных CSS-правил.

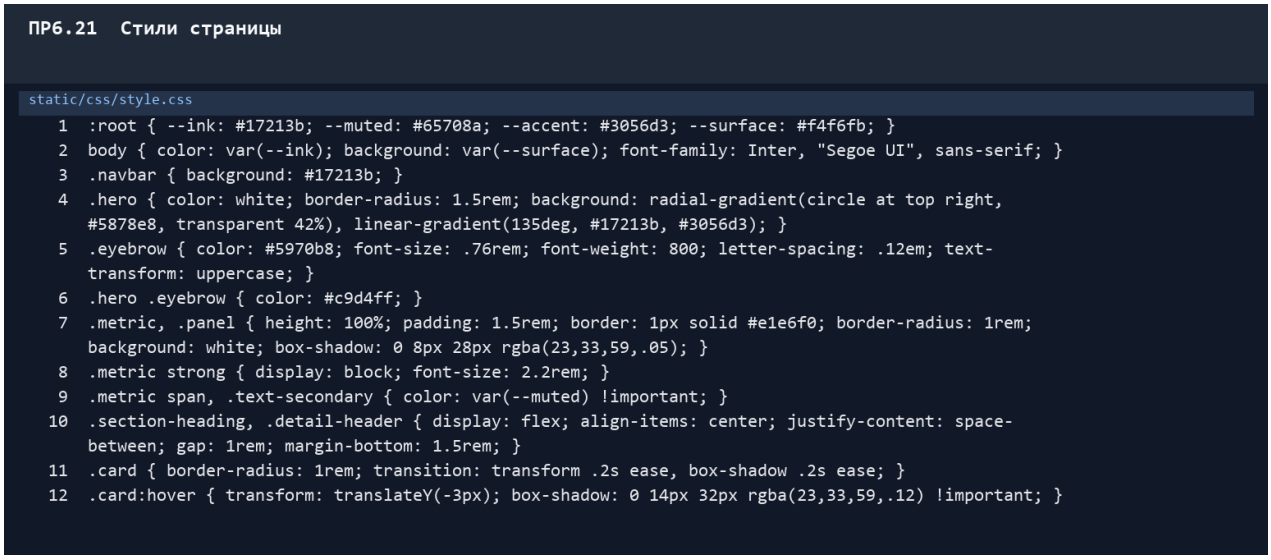


Рисунок 621 – Подтверждение выполнения пункта 21

ПР6.22 Диагностика подключения JavaScript

Полное условие: Покажите подключение JavaScript-файла и элемент страницы, с которым он взаимодействует. Объясните, как отличить отсутствие реакции из-за ошибки в скрипте от ситуации, когда файл вообще не загрузился.

Краткий ответ: Файл должен загрузиться без 404, затем ошибки выполнения проверяются в Console; это разные причины отсутствия реакции.

Подтверждение в проекте: Показан подключённый файл static/js/main.js.

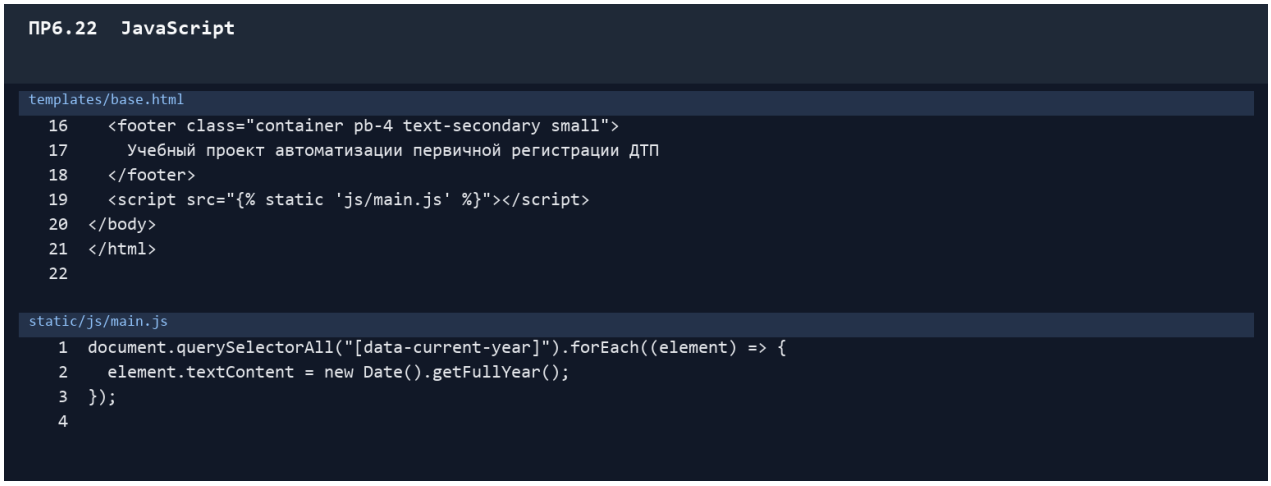


Рисунок 622 – Подтверждение выполнения пункта 22

ПР6.23 Организация шаблонов приложений

Полное условие: Покажите расположение шаблонов нескольких приложений Roomly и путь к одному из них в render(). Объясните, почему структура каталогов и уникальные пути шаблонов помогают избежать конфликтов имён.

Краткий ответ: Шаблоны разнесены по пространствам homepage/... и accidents/..., что предотвращает совпадение имён.

Подтверждение в проекте: Дерево проекта показывает раздельные каталоги шаблонов приложений.

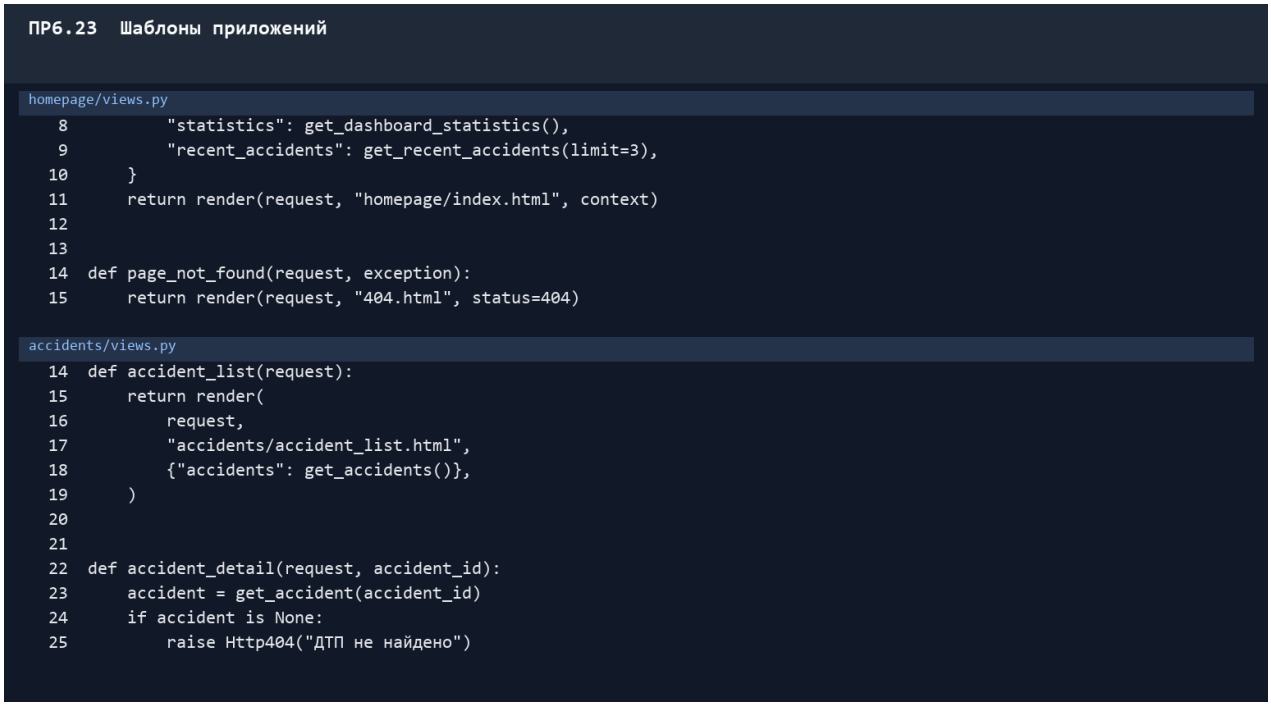


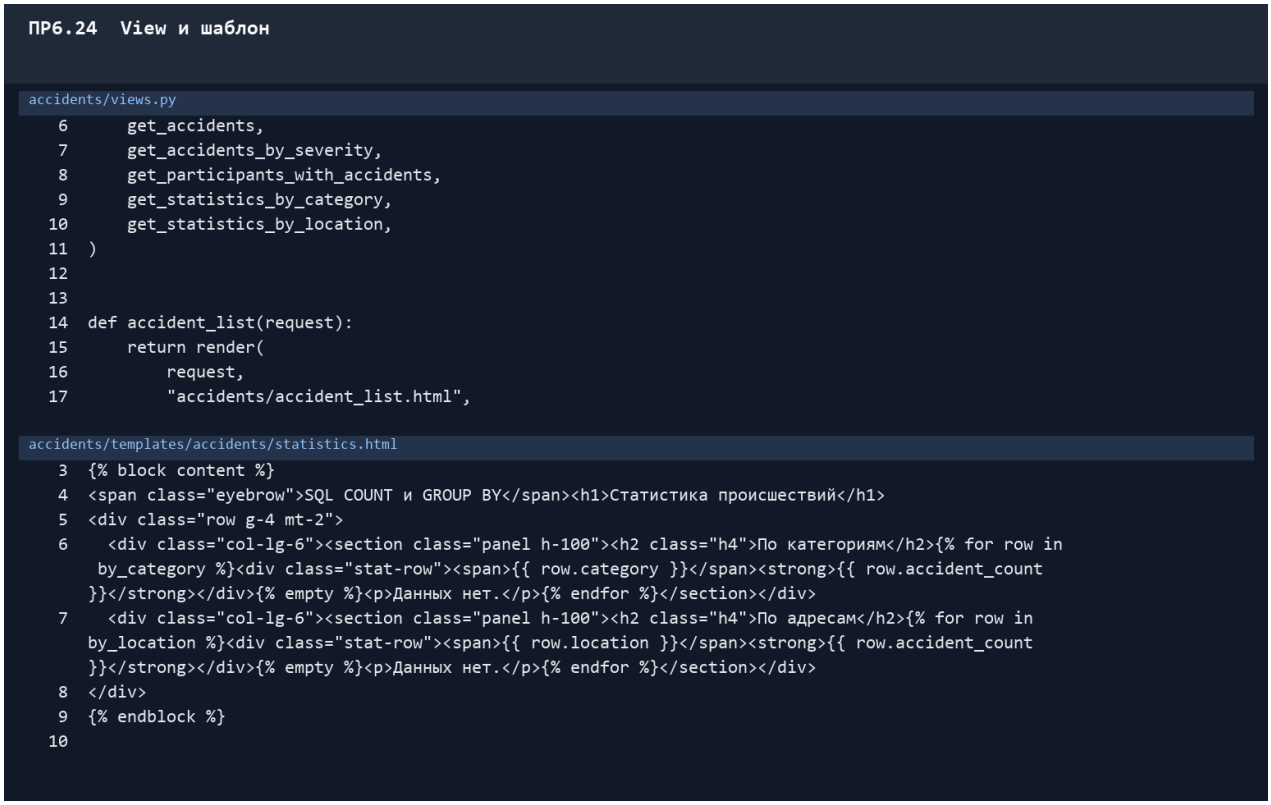
Рисунок 623 – Подтверждение выполнения пункта 23

ПР6.24 Согласованность представления и шаблона

Полное условие: Покажите, как представление передаёт объект в шаблон и как шаблон использует его атрибуты. Измените имя ключа контекста или обращение к атрибуту, продемонстрируйте возникшую проблему и исправьте её.

Краткий ответ: Ключи context должны совпадать с переменными шаблона; исправляется одна из двух сторон соглашения.

Подтверждение в проекте: Ключи контекста во view согласованы с переменными шаблона.



ПР6.24 View и шаблон

accidents/views.py

```
    get_accidents,
    get_accidents_by_severity,
    get_participants_with_accidents,
    get_statistics_by_category,
    get_statistics_by_location,
)


def accident_list(request):
    return render(
        request,
        "accidents/accident_list.html",
```

accidents/templates/accidents/statistics.html

```
{% block content %}
<span class="eyebrow">SQL COUNT и GROUP BY</span><h1>Статистика происшествий</h1>
<div class="row g-4 mt-2">
  <div class="col-lg-6"><section class="panel h-100"><h2 class="h4">По категориям</h2>{% for row in
 by_category %}<div class="stat-row"><span>{{ row.category }}</span><strong>{{ row.accident_count
}}</strong></div>{% empty %}<p>Данных нет.</p>{% endfor %}</section></div>
  <div class="col-lg-6"><section class="panel h-100"><h2 class="h4">По адресам</h2>{% for row in
by_location %}<div class="stat-row"><span>{{ row.location }}</span><strong>{{ row.accident_count
}}</strong></div>{% empty %}<p>Данных нет.</p>{% endfor %}</section></div>
</div>
{% endblock %}
```

Рисунок 624 – Подтверждение выполнения пункта 24

ПР6.25 Путь обработки HTTP-запроса

Полное условие: Проследите путь запроса к странице конкретной комнаты, например /rooms/1/, от URL-маршрута до отображения HTML. Покажите соответствующие маршрут, представление и шаблон, объясните передачу идентификатора и данных между этапами. Укажите, на каком этапе возникнет ошибка, если комната с таким идентификатором не существует.

Краткий ответ: Запрос проходит URLconf → accident_detail → SQL → context → accident_detail.html; неизвестный id даёт Http404 во view.

Подтверждение в проекте: Показан итог полного пути /accidents/2/ до HTML.

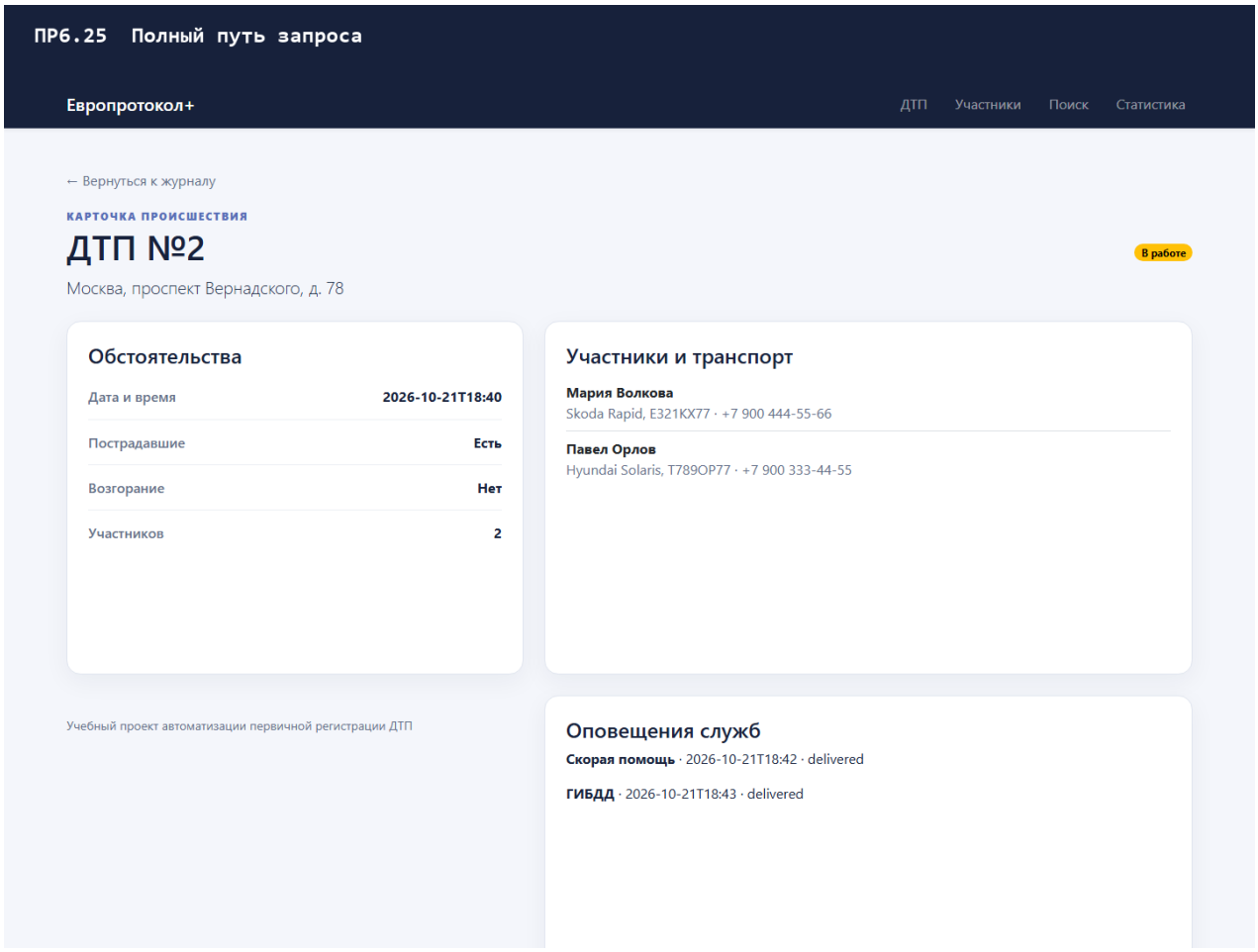


Рисунок 625 – Подтверждение выполнения пункта 25

ПР7 ПОЛНАЯ ТАБЛИЦА ДЕМОНСТРАЦИИ

Каждый пункт раздела содержит полное условие из таблицы оценивания, краткий ответ, пояснение по проекту «Европротокол+» и точечный скриншот подтверждения.

ПР7.1 Проектирование структуры базы данных

Полное условие: Покажите структуру базы данных своего проекта. Объясните назначение основных таблиц, состав хранимых данных и связи между сущностями предметной области.

Краткий ответ: accidents хранит ДТП, vehicles — автомобили, participants связывает ДТП и автомобиль, emergency_messages хранит оповещения.

Подтверждение в проекте: Показана полная схема четырёх таблиц предметной области.

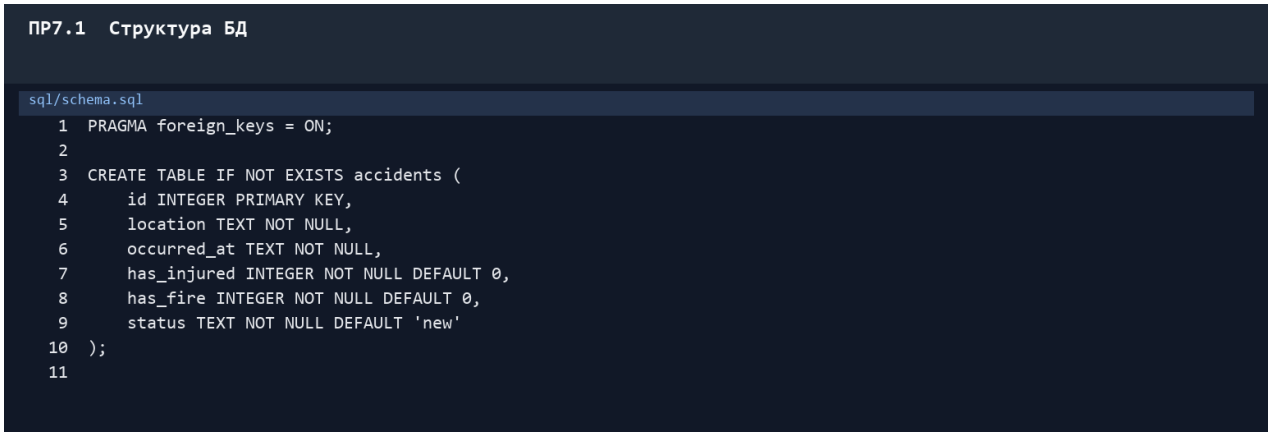


Рисунок 701 – Подтверждение выполнения пункта 1

ПР7.2 Создание таблиц и связей

Полное условие: Покажите SQL-команды создания таблиц. Объясните назначение первичных и внешних ключей и продемонстрируйте, как они обеспечивают связь записей между таблицами.

Краткий ответ: PRIMARY KEY уникально определяет строку, FOREIGN KEY связывает зависимую запись с существующим ДТП или автомобилем.

Подтверждение в проекте: В schema.sql видны PRIMARY KEY, FOREIGN KEY и связи.



Рисунок 702 – Подтверждение выполнения пункта 2

ПР7.3 Создание базы данных и выполнение схемы

Полное условие: Продемонстрируйте создание базы SQLite и выполнение скрипта создания таблиц. Объясните, как организовать повторный запуск скрипта, чтобы избежать ошибок при наличии существующих таблиц.

Краткий ответ: init_db выполняет schema.sql; IF NOT EXISTS позволяет безопасно запускать схему повторно.

Подтверждение в проекте: init_db.py выполняет схему с CREATE TABLE IF NOT EXISTS.

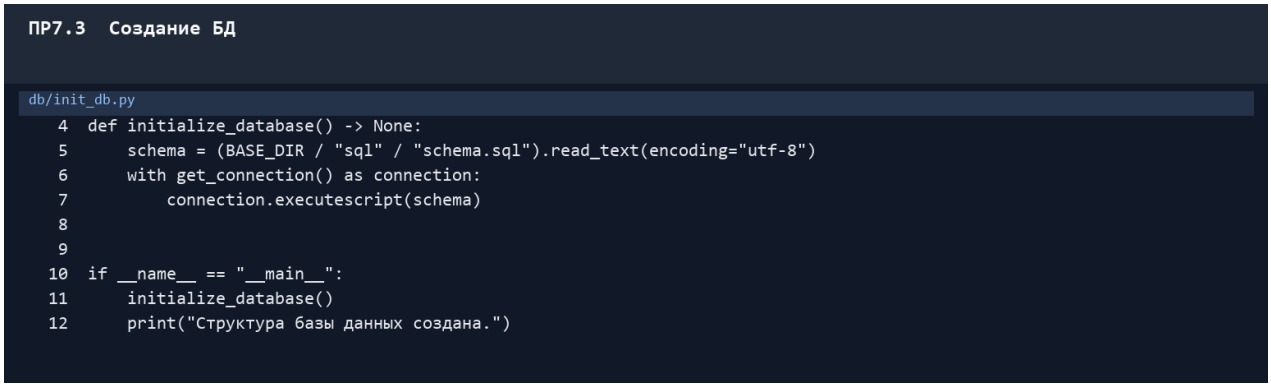


Рисунок 703 – Подтверждение выполнения пункта 3

ПР7.4 Подключение к базе данных

Полное условие: Продемонстрируйте подключение Python к файлу SQLite и выполнение простого SQL-запроса. Объясните, как убедиться, что приложение использует нужную базу данных.

Краткий ответ: get_connection открывает data/europrotocol.sqlite, включает внешние ключи и именованные строки sqlite3.Row.

Подтверждение в проекте: database.py задаёт абсолютный путь, foreign_keys и sqlite3.Row.

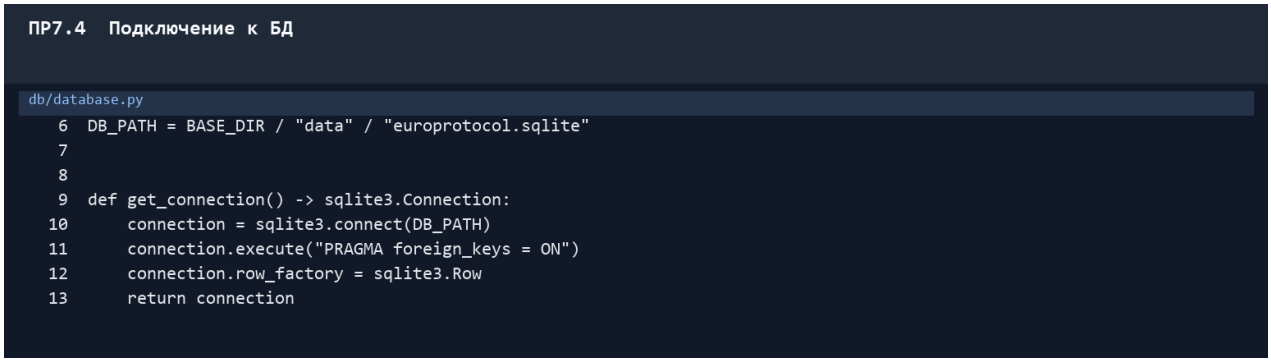


Рисунок 704 – Подтверждение выполнения пункта 4

ПР7.5 Импорт данных из JSON

Полное условие: Продемонстрируйте перенос данных из JSON-файлов в таблицы SQLite. Объясните, как преобразуются исходные данные и почему при наличии связей между записями важен порядок импорта.

Краткий ответ: Импорт читает JSON и вставляет сначала независимые accidents/vehicles, затем participants/messages с внешними ключами.

Подтверждение в проекте: import_json.py переносит четыре JSON-набора в правильном порядке.



Рисунок 705 – Подтверждение выполнения пункта 5

ПР7.6 Проверка результатов импорта

Полное условие: Выполните SQL-запросы для подсчёта записей в таблицах. Сравните результаты с исходными JSON-файлами и объясните, как выявить неполный импорт данных.

Краткий ответ: COUNT по каждой таблице сравнивается с длиной соответствующего JSON-массива; расхождение показывает неполный импорт.

Подтверждение в проекте: Фактический вывод подтверждает импорт и успешные проверки.

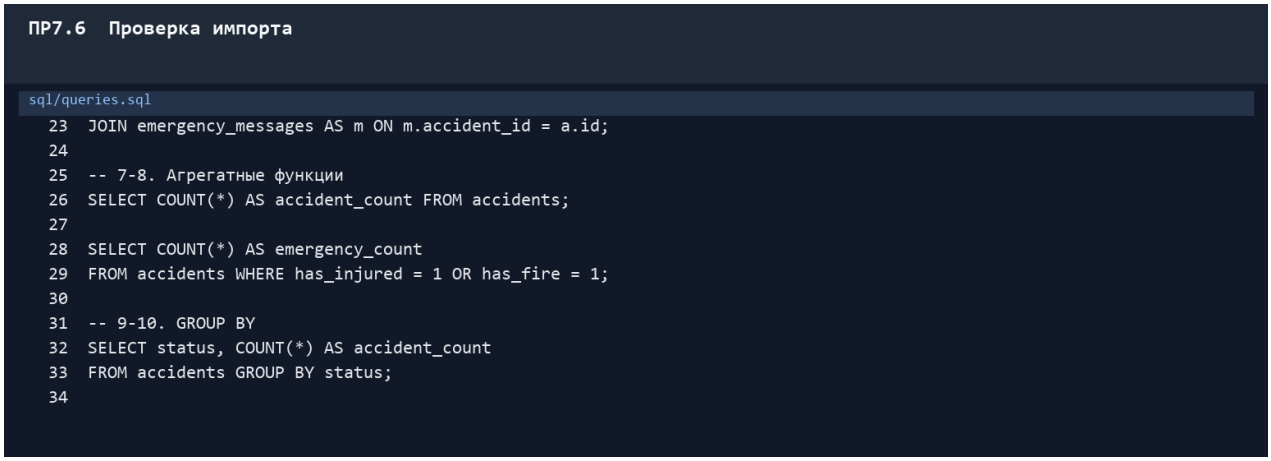


Рисунок 706 – Подтверждение выполнения пункта 6

ПР7.7 Выборка данных с помощью SELECT

Полное условие: Покажите SQL-запрос для получения списка объектов основной сущности проекта. Измените состав выбираемых полей и объясните, как это влияет на результат запроса и данные, доступные для отображения.

Краткий ответ: SELECT задаёт список возвращаемых столбцов; удалённое поле больше недоступно view и шаблону.

Подтверждение в проекте: В queries.sql показаны два запроса выборки списков.



Рисунок 707 – Подтверждение выполнения пункта 7

ПР7.8 Фильтрация данных с помощью WHERE

Полное условие: Покажите SQL-запрос для поиска записей по заданному условию. Измените условие отбора и продемонстрируйте, как оно влияет на состав результата.

Краткий ответ: WHERE отбирает строки до выдачи результата; изменение условия меняет состав карточек.

Подтверждение в проекте: Страница поиска показывает результат WHERE.

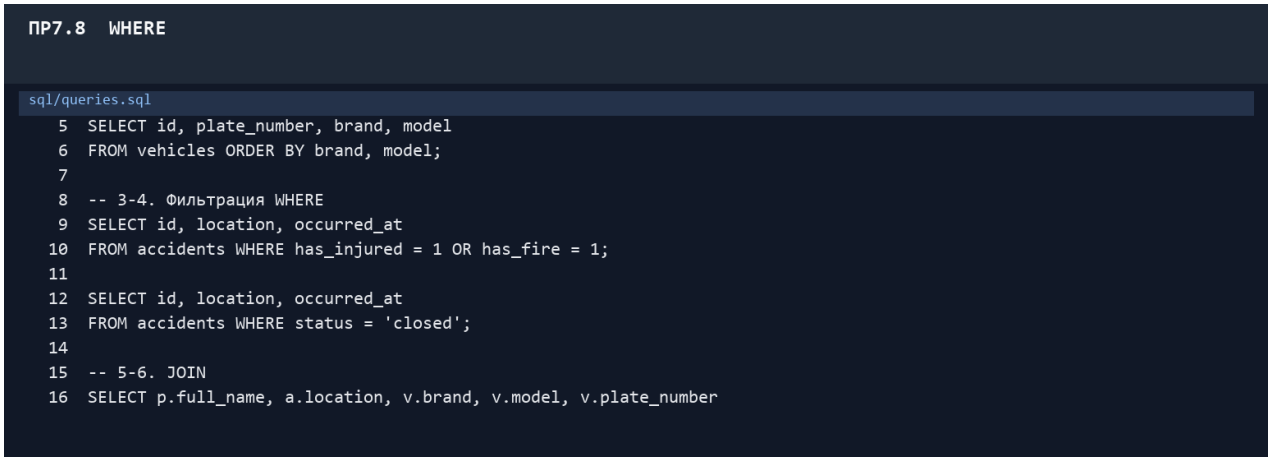


Рисунок 708 – Подтверждение выполнения пункта 8

ПР7.9 Фильтрация записей по состоянию или признаку

Полное условие: Покажите SQL-запрос для получения записей, соответствующих определённому состоянию или признаку предметной области. Измените условие и объясните, какие записи включаются в результат и какие исключаются.

Краткий ответ: Условие status='closed' включает только закрытые ДТП; другой статус формирует другой набор.

Подтверждение в проекте: В SQL есть фильтрация по status и признакам опасности.

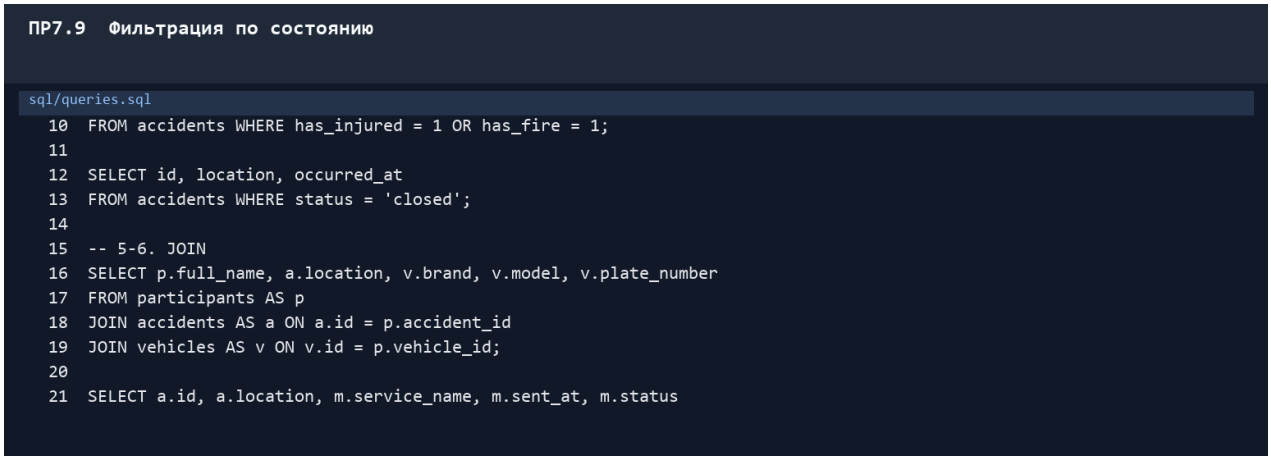


Рисунок 709 – Подтверждение выполнения пункта 9

ПР7.10 Объединение таблиц с помощью JOIN

Полное условие: Покажите SQL-запрос, объединяющий данные из нескольких связанных таблиц. Объясните условия соединения и продемонстрируйте, как запрос позволяет получить сведения о связанных сущностях.

Краткий ответ: JOIN связывает participant.accident_id/vehicle_id с первичными ключами и выдаёт данные нескольких сущностей одной строкой.

Подтверждение в проекте: Страница участников показывает JOIN трёх таблиц.

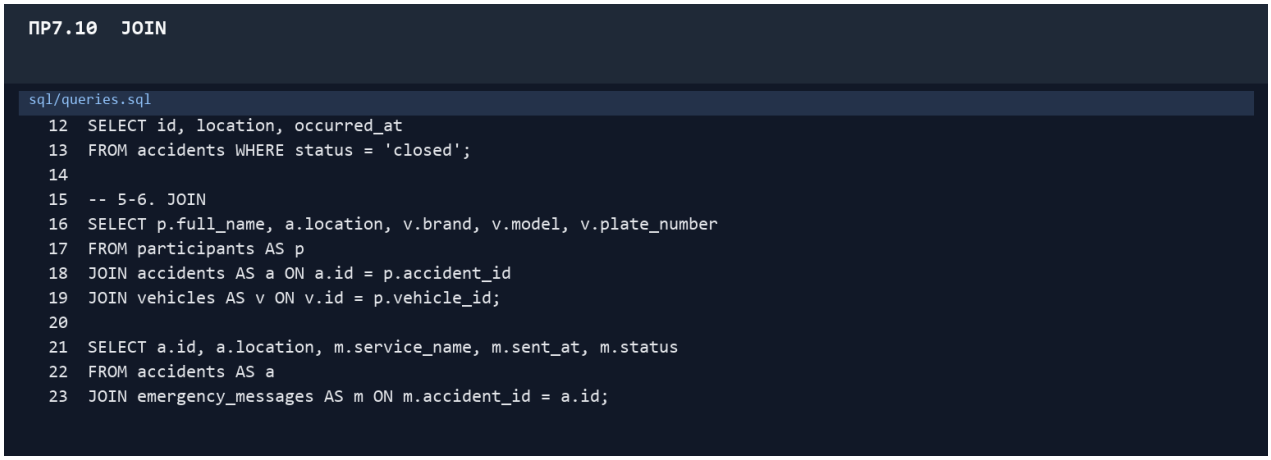


Рисунок 710 – Подтверждение выполнения пункта 10

ПР7.11 Отбор данных по нескольким условиям

Полное условие: Покажите SQL-запрос, который выбирает записи с учётом нескольких условий предметной области, включая при необходимости дату или другой параметр. Измените одно из условий и объясните, как изменится результат.

Краткий ответ: OR включает ДТП с любым опасным признаком, AND потребовал бы одновременного выполнения обоих условий.

Подтверждение в проекте: Показан запрос с несколькими условиями has_injured OR has_fire.



Рисунок 711 – Подтверждение выполнения пункта 11

ПР7.12 Подсчёт записей с помощью COUNT()

Полное условие: Покажите SQL-запрос для подсчёта количества записей, соответствующих задаче проекта. Объясните назначение COUNT() и результат запроса при отсутствии подходящих записей.

Краткий ответ: COUNT считает подходящие строки и возвращает 0, если совпадений нет.

Подтверждение в проекте: Статистика отображает результаты COUNT.

ПР7.12 COUNT

db/queries.py

```
139 def get_dashboard_statistics() -> dict:
140     return fetch_one(
141         """
142         SELECT COUNT(*) AS total,
143                SUM(CASE WHEN has_injured = 1 OR has_fire = 1
144                         THEN 1 ELSE 0 END) AS emergencies,
145                SUM(CASE WHEN status = 'closed' THEN 1 ELSE 0 END) AS closed
146         FROM accidents
147         """
148     ) or {"total": 0, "emergencies": 0, "closed": 0}
```

Рисунок 712 – Подтверждение выполнения пункта 12

ПР7.13 Группировка данных с помощью GROUP BY

Полное условие: Покажите SQL-запрос для подсчёта или обобщения данных по категориям либо связанным сущностям. Объясните назначение GROUP BY и то, как формируются отдельные группы результатов.

Краткий ответ: GROUP BY формирует группы категорий или адресов, а COUNT вычисляется отдельно для каждой группы.

Подтверждение в проекте: Статистика сгруппирована по категориям и адресам.



Рисунок 713 – Подтверждение выполнения пункта 13

ПР7.14 Фильтрация групп с помощью HAVING

Полное условие: Покажите SQL-запрос, который отбирает группы по результатам агрегатных вычислений. Измените пороговое значение и объясните отличие HAVING от WHERE.

Краткий ответ: WHERE фильтрует строки до группировки, HAVING — уже сформированные группы после агрегирования.

Подтверждение в проекте: В get_statistics_by_location используется HAVING.

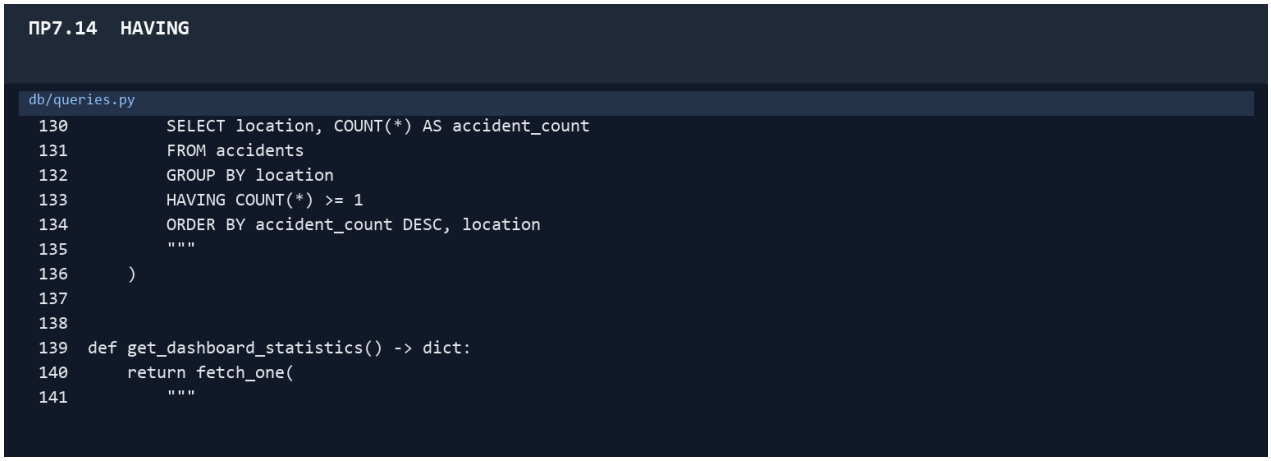


Рисунок 714 – Подтверждение выполнения пункта 14

ПР7.15 Параметризованные SQL-запросы

Полное условие: Покажите SQL-запрос, условие которого зависит от значения, полученного от пользователя или переданного приложением. Продемонстрируйте передачу значения отдельным параметром и объясните, почему не следует подставлять пользовательский ввод непосредственно в строку SQL.

Краткий ответ: Пользовательское значение передаётся отдельным параметром ?, что предотвращает SQL-инъекцию и ошибки экранирования.

Подтверждение в проекте: Параметры передаются отдельно через ? и tuple, а не склейкой строки.

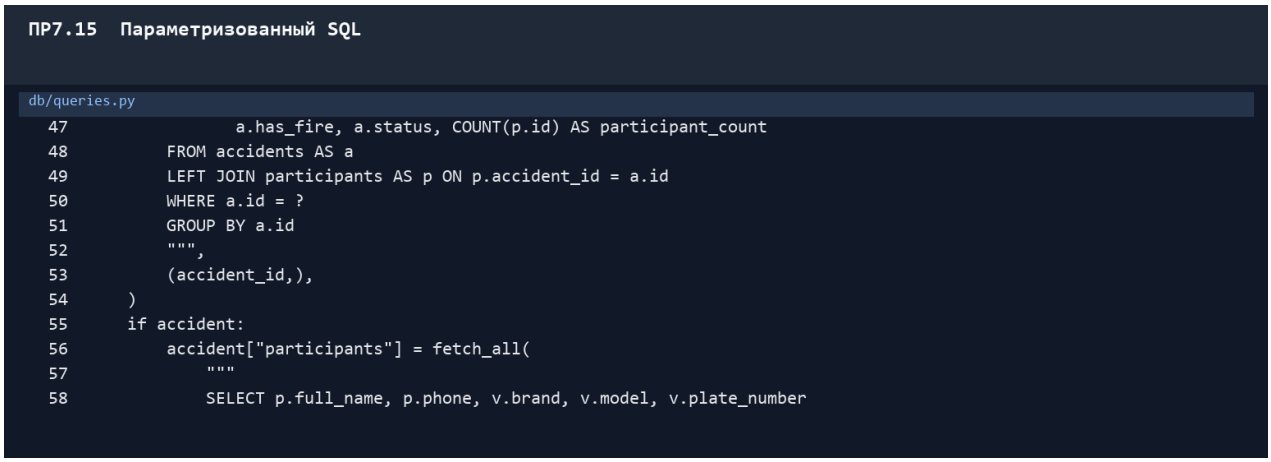


Рисунок 715 – Подтверждение выполнения пункта 15

ПР7.16 Получение и обработка результатов SQL

Полное условие: Продемонстрируйте выполнение SQL-запроса из Python-кода Django и обработку полученных строк. Покажите обращение к полям результата по именам столбцов и объясните назначение соответствующей настройки SQLite.

Краткий ответ: sqlite3.Row даёт доступ по имени столбца; затем строки преобразуются в dict и передаются дальше.

Подтверждение в проекте: sqlite3.Row позволяет обращаться к полям по именам.

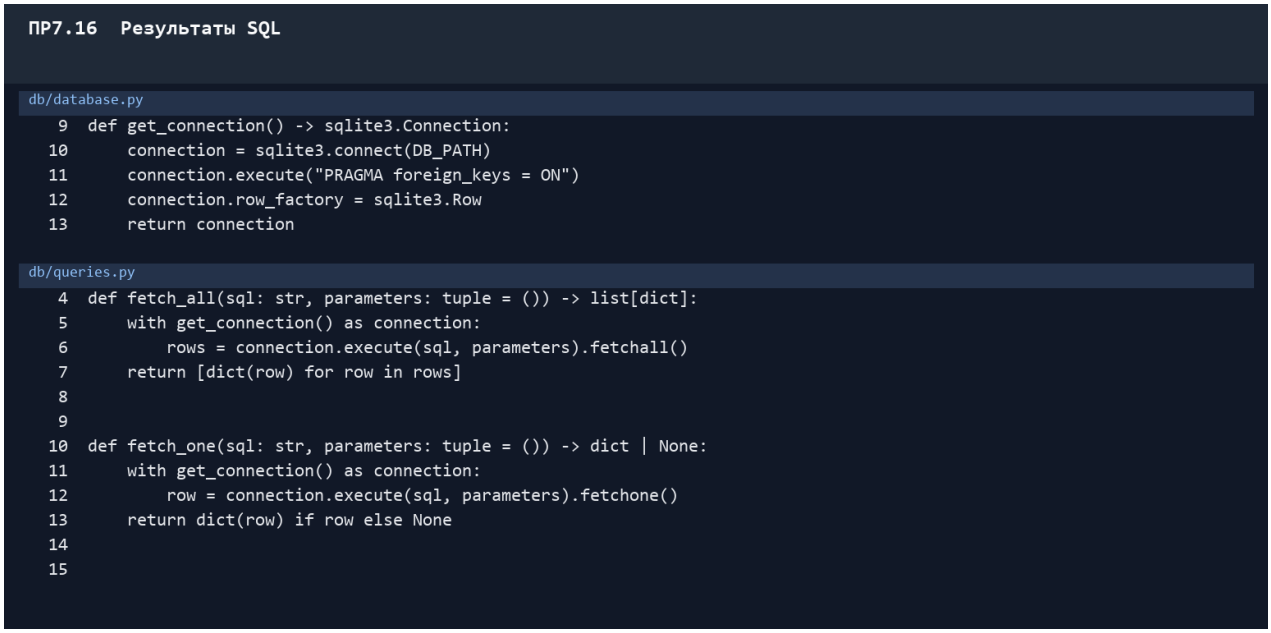


Рисунок 716 – Подтверждение выполнения пункта 16

ПР7.17 Отображение списка объектов

Полное условие: Продемонстрируйте страницу со списком объектов основной сущности проекта, данные для которой получены из SQLite. Объясните, как результат SQL-запроса передаётся из представления в шаблон.

Краткий ответ: get_accidents читает SQLite, view передаёт accidents в context, цикл шаблона формирует журнал.

Подтверждение в проекте: Журнал ДТП показывает список, полученный из SQLite.

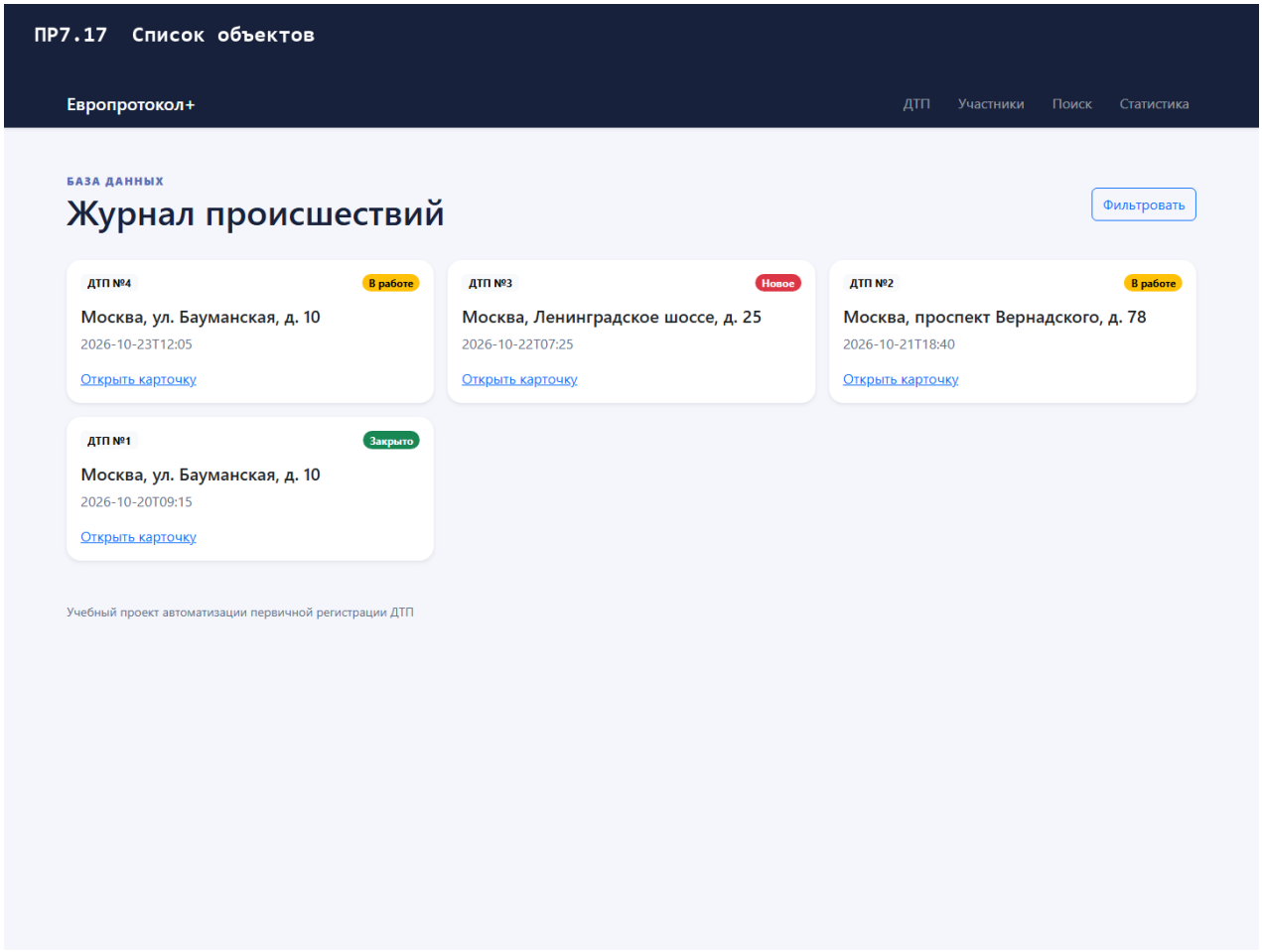


Рисунок 717 – Подтверждение выполнения пункта 17

ПР7.18 Отображение отфильтрованных записей

Полное условие: Продемонстрируйте страницу, отображающую записи, отобранные по условию предметной области. Объясните, как SQL-фильтрация определяет состав отображаемых данных.

Краткий ответ: SQL-фильтр определяет записи до передачи в шаблон, поэтому страница показывает только совпадения.

Подтверждение в проекте: Фильтр показывает отобранные SQL-запросом записи.

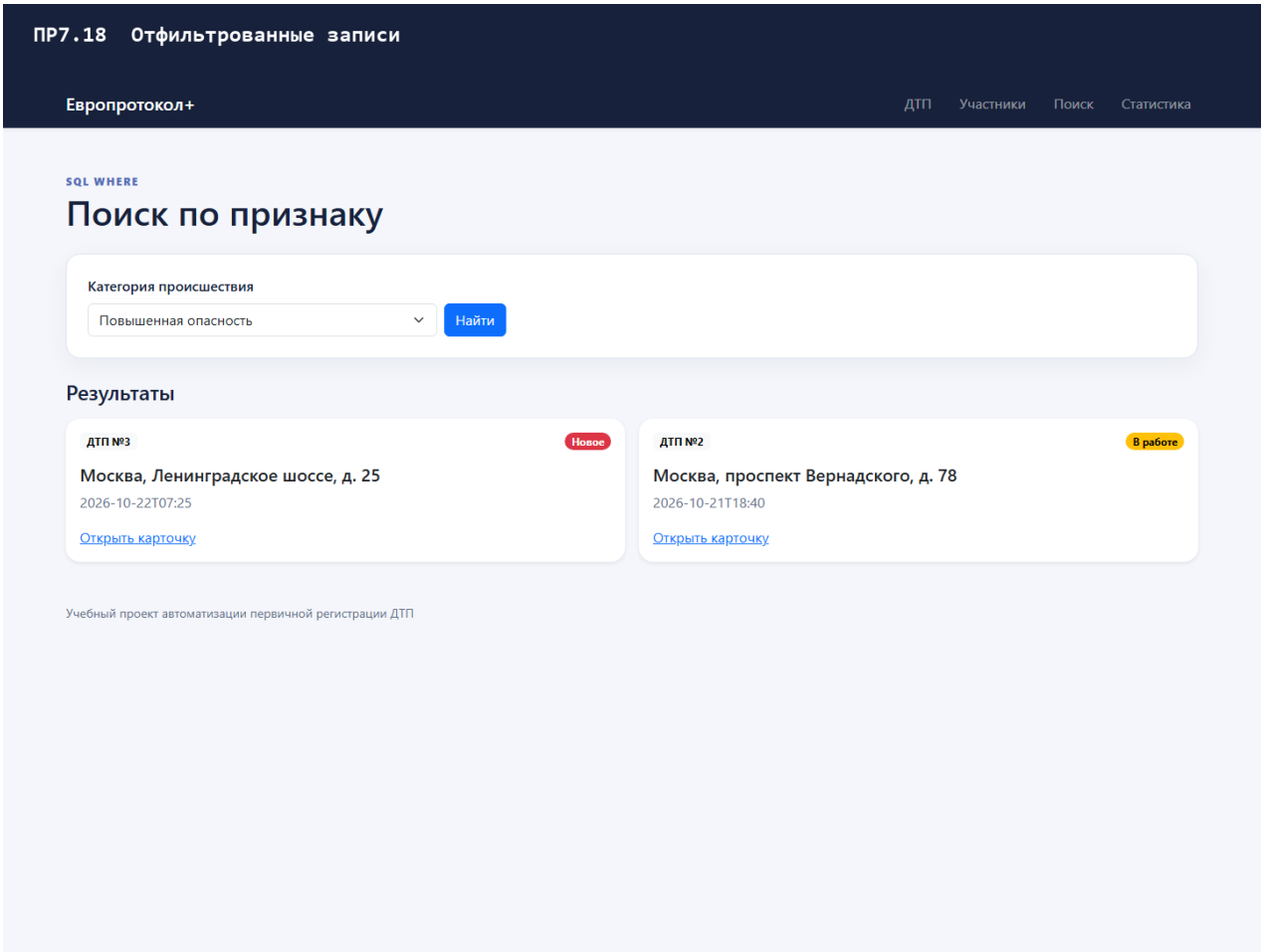


Рисунок 718 – Подтверждение выполнения пункта 18

ПР7.19 Отображение связанных данных

Полное условие: Продемонстрируйте страницу, на которой одновременно представлены сведения о нескольких связанных сущностях. Объясните, как результат JOIN используется при формировании страницы.

Краткий ответ: JOIN объединяет ФИО, ДТП и автомобиль; результат напрямую используется таблицей страницы участников.

Подтверждение в проекте: JOIN одновременно выводит участника, автомобиль и ДТП.

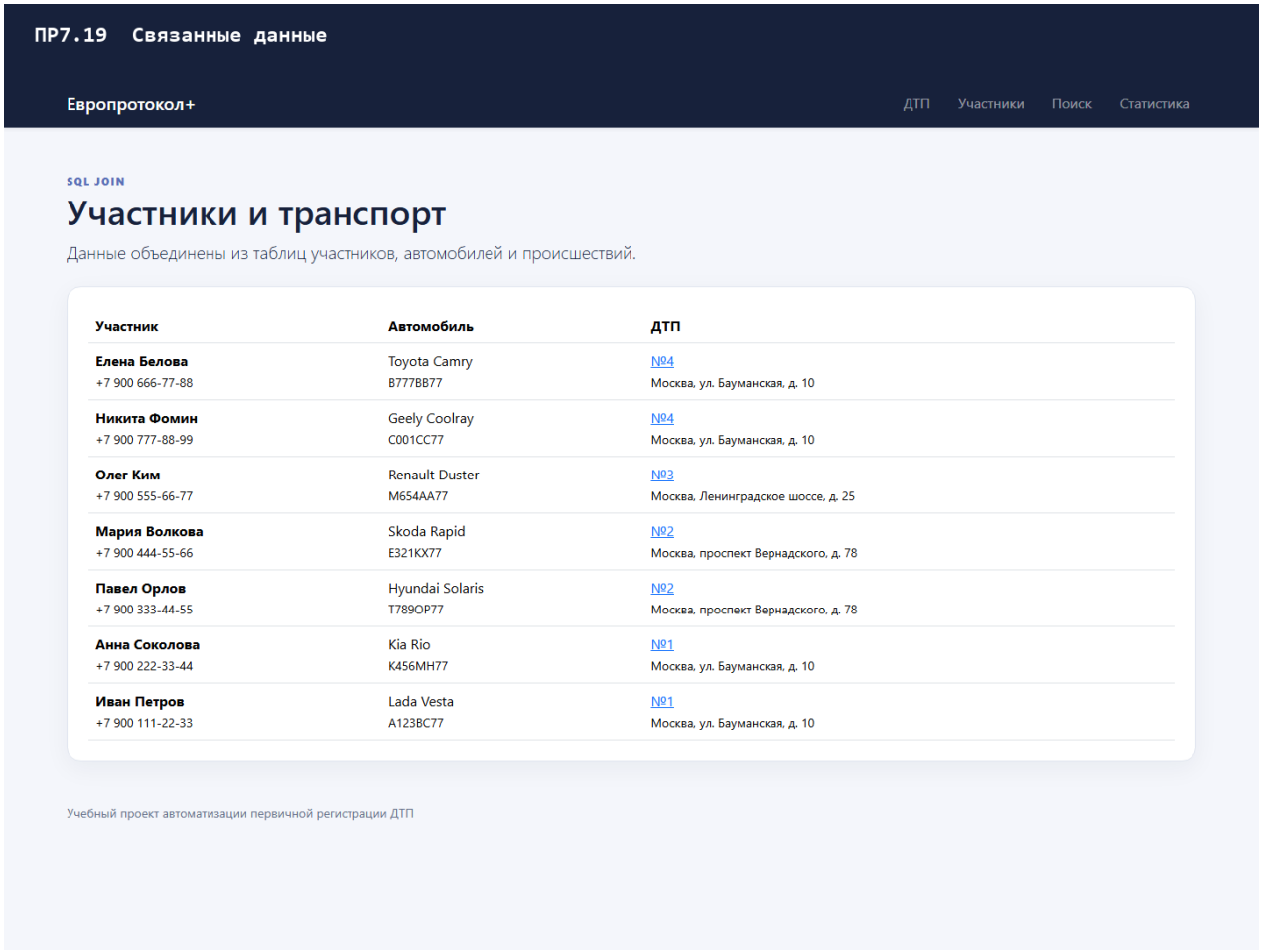


Рисунок 719 – Подтверждение выполнения пункта 19

ПР7.20 Реализация поиска по пользовательским параметрам

Полное условие: Продемонстрируйте страницу поиска или фильтрации данных по значению, введённому или выбранному пользователем. Покажите передачу параметра в представление и выполнение соответствующего SQL-запроса.

Краткий ответ: GET severity читается во view и выбирает разрешённое условие; произвольный текст не вставляется в SQL.

Подтверждение в проекте: GET-параметр severity проходит view и выбирает безопасное условие.



Рисунок 720 – Подтверждение выполнения пункта 20

ПР7.21 Отображение статистических данных

Полное условие: Продемонстрируйте страницу со статистикой по данным проекта. Объясните, как результаты агрегатных запросов передаются в шаблон и отображаются пользователю.

Краткий ответ: Агрегатные запросы возвращают by_category и by_location, которые выводятся отдельными циклами.

Подтверждение в проекте: Страница статистики показывает агрегированные данные.

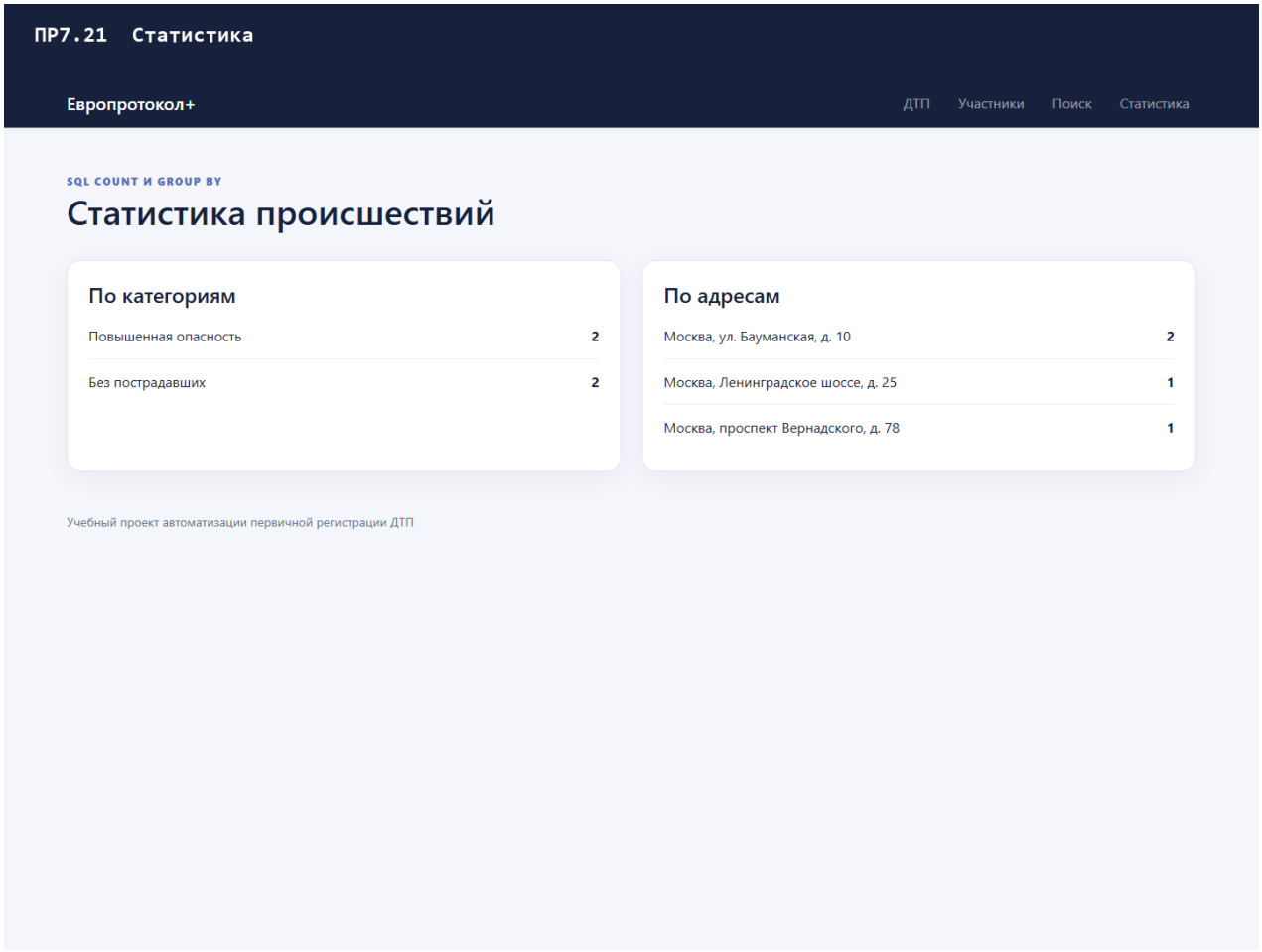


Рисунок 721 – Подтверждение выполнения пункта 21

ПР7.22 Передача результатов в шаблон

Полное условие: Продемонстрируйте исправление ситуации, когда SQL-запрос возвращает данные, но они не отображаются из-за несоответствия имён переменных в контексте представления и шаблоне.

Краткий ответ: Имена by_category/by_location должны совпадать во view и шаблоне; иначе данные есть, но переменная пуста.

Подтверждение в проекте: Во view видны ключи context, совпадающие с шаблоном.

ПР7.22 Результаты в шаблоне

accidents/views.py

```
    get_accidents,
    get_accidents_by_severity,
    get_participants_with_accidents,
    get_statistics_by_category,
    get_statistics_by_location,
)


def accident_list(request):
    return render(
        request,
        "accidents/accident_list.html",
```

accidents/templates/accidents/statistics.html

```
{% block content %}
<span class="eyebrow">SQL COUNT и GROUP BY</span><h1>Статистика происшествий</h1>
<div class="row g-4 mt-2">
  <div class="col-lg-6"><section class="panel h-100"><h2 class="h4">По категориям</h2>{% for row in by_category %}<div class="stat-row"><span>{{ row.category }}</span><strong>{{ row.accident_count }}</strong></div>{% empty %}<p>Данных нет.</p>{% endfor %}</section></div>
  <div class="col-lg-6"><section class="panel h-100"><h2 class="h4">По адресам</h2>{% for row in by_location %}<div class="stat-row"><span>{{ row.location }}</span><strong>{{ row.accident_count }}</strong></div>{% empty %}<p>Данных нет.</p>{% endfor %}</section></div>
</div>
{% endblock %}
```

Рисунок 722 – Подтверждение выполнения пункта 22

ПР7.23 Обработка пустых результатов

Полное условие: Продемонстрируйте поведение страницы, если по заданным условиям не найдено ни одной записи. Объясните, чем пустой результат запроса отличается от ошибки выполнения SQL.

Краткий ответ: Пустой список является корректным результатом SELECT и обрабатывается empty; ошибка SQL прерывает выполнение исключением.

Подтверждение в проекте: Пустой результат выводится как «записей нет» без ошибки SQL.



Рисунок 723 – Подтверждение выполнения пункта 23

ПР7.24 Диагностика ошибок SQL-запросов

Полное условие: Разберите пример некорректного результата, вызванного ошибкой в имени поля, условии отбора или соединении таблиц. Исправьте запрос и продемонстрируйте корректный результат в приложении.

Краткий ответ: Диагностика идёт поэтапно: имя поля, условие, ON-связь, параметры, результат функции и только затем шаблон.

Подтверждение в проекте: Показаны SQL, условия JOIN и тесты для диагностики.

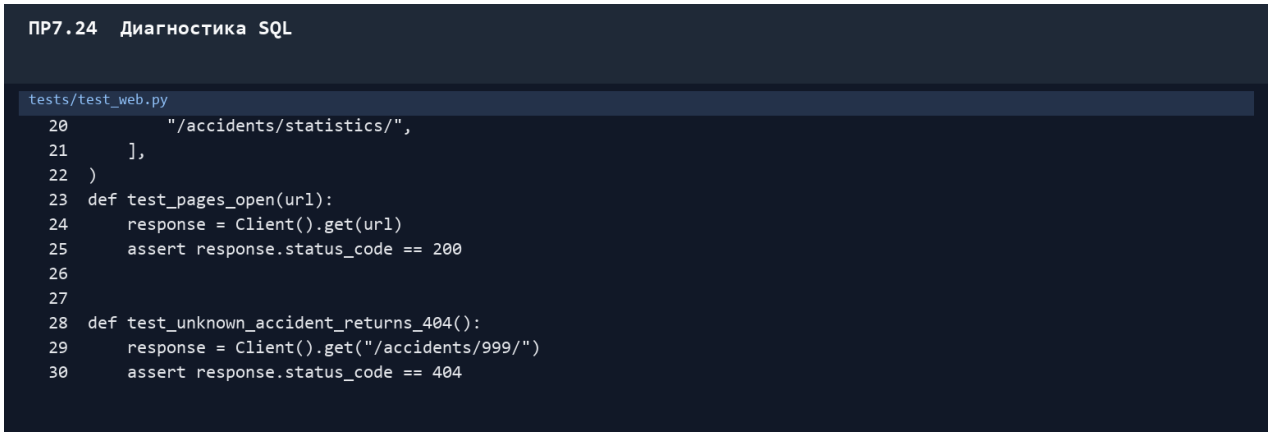


Рисунок 724 – Подтверждение выполнения пункта 24

ПР7.25 Сквозная обработка запроса в Django

Полное условие: Проследите полный путь обработки пользовательского запроса: получение параметров, выполнение SQL, обработка результатов в Django и отображение данных в браузере. Объясните, на каких этапах следует искать причину некорректного результата.

Краткий ответ: Полный путь: GET severity → URL → accident_search → параметризованный SQL → context → цикл шаблона.

Подтверждение в проекте: Сквозной путь: GET → URL → view → SQL → context → HTML.



Рисунок 725 – Подтверждение выполнения пункта 25